Relation Algebra

§1. Notation

Traditionaly Adjunction F -| G is defined as $F(x) \leq y$ iff $x \leq G(y)$, and you derive laws like this:

$$\begin{aligned} F(x) \leq F(x) &\implies x \leq G(F(x)) \implies F(x) \leq F(G(F(x))) \\ G(y) \leq G(y) &\implies F(G(y)) \leq y \implies G(F(G(y))) \leq G(y) \end{aligned}$$

By define $X : * \mapsto x$, $Y : * \mapsto y$ and use diagram order, application can be replaced by composition: $XF \leq Y$ iff $X \leq YG$

(1a)

- write F as $/$, G as $\backslash$
- render $a \leq b$ as $\boxed{\begin{matrix} a \\ \hline b \end{matrix}}$
- **rewrite** $X \leq XF$ as $1 \leq F$

$$\boxed{\begin{matrix} X/ \\ \hline Y \end{matrix}} = \boxed{\begin{matrix} X \\ \hline Y\backslash \end{matrix}} \qquad \text{adj}$$

$$\boxed{\begin{matrix} X/ \\ \hline X/ \end{matrix}} \overset{\text{adj}}{\implies} \boxed{\begin{matrix} X \\ \hline X/\backslash \end{matrix}} \overset{\text{rewrite}}{=\!=} \underset{\text{unit}}{\boxed{\begin{matrix} * \\ \hline /\backslash \end{matrix}}} \quad \begin{matrix} \overset{\text{put } \backslash \text{ on left}}{\implies} \boxed{\begin{matrix} \backslash \\ \hline \backslash/\backslash \end{matrix}} \\ \overset{\text{put } / \text{ on right}}{\implies} \boxed{\begin{matrix} / \\ \hline /\backslash/ \end{matrix}} \end{matrix}$$

$$\boxed{\begin{matrix} Y\backslash \\ \hline Y\backslash \end{matrix}} \overset{\text{adj}}{\implies} \boxed{\begin{matrix} Y\backslash/ \\ \hline Y \end{matrix}} \overset{\text{rewrite}}{=\!=} \underset{\text{counit}}{\boxed{\begin{matrix} \backslash/ \\ \hline * \end{matrix}}} \quad \begin{matrix} \overset{\text{put } / \text{ on left}}{\implies} \boxed{\begin{matrix} /\backslash/ \\ \hline / \end{matrix}} \\ \overset{\text{put } \backslash \text{ on right}}{\implies} \boxed{\begin{matrix} \backslash/\backslash \\ \hline \backslash \end{matrix}} \end{matrix}$$

now we have $/\backslash/ = /$ and $\backslash/\backslash = \backslash$, and you just discovered string diagrams.

§1.1. Adjunctions

(1.1a)

F ⊣ G	F's type	monad FG	𝟙⊑FG	GF⊑𝟙	F preserves ∪	G preserves ∩	FGF=F	GFG=G
◁ ⊣ ▷	A → A⊗A	◁▷=𝟙	𝟙⊑◁▷	▷◁⊑𝟙	(R∪S)◁= R◁∪S◁	(R∩S)▷= R▷∩S▷	◁▷◁=◁	▷◁▷=▷
⊸ ⊣ ⟜	A → 𝕀	⊸⟜=⊤	𝟙⊑⊸⟜	⟜⊸⊑𝟙	(R∪S)⊸= R⊸∪S⊸	(R∩S)⟜= R⟜∩S⟜	⊸⟜⊸=⊸	⟜⊸⟜=⟜
° ⊣ °	(A⇸B) → (B⇸A)	°°=𝟙	R=R°°	R=R°°	(R∪S)°= R°∪S°	(R∩S)°= R°∩S°	R°°°=R°	R°°°=R°
⟜◁ ⊣ ▷⊸	(X⊗A → Y) → (X → Y⊗A)	𝟙	(⟜◁⊗𝟙) (𝟙⊗▷⊸)=𝟙	(𝟙⊗⟜◁) (▷⊸⊗𝟙)=𝟙	=	=	=	=
▷⊸ ⊣ ⟜◁	(X → Y⊗A) → (X⊗A → Y)	𝟙	(𝟙⊗⟜◁) (▷⊸⊗𝟙)=𝟙	(⟜◁⊗𝟙) (𝟙⊗▷⊸)=𝟙	=	=	=	=
Δ ⊣ ∩	(A⇸B) → (A⇸B)²	R↦R∩R=R	R⊑R∩R	R∩S⊑R	Δ(R∪S)= ΔR∪ΔS	(R∩T)∩(S∩U)= (R∩S)∩(T∩U)	R∩R=R	R∩R=R
∪ ⊣ Δ	(A⇸B)² → (A⇸B)	(R,S)↦ (R∪S,R∪S)	R⊑R∪S	R∪R⊑R	(R∪T)∪(S∪U)= (R∪S)∪(T∪U)	Δ(R∩S)= ΔR∩ΔS	R∪R=R	R∪R=R
⊥ ⊣ !	{*} → (A⇸B)	—	—	⊥⊑R	—	—	—	—
·S ⊣ /S	(A⇸B) → (A⇸C)	S/S	R⊑(RS)/S	(T/S)S⊑T	(R∪T)S= RS∪TS	(R∩T)/S= R/S∩T/S	((RS)/S)S =RS	((T/S)S)/S =T/S
S· ⊣ S\	(B⇸C) → (A⇸C)	S\S	R⊑S\(SR)	S(S\T)⊑T	S(R∪T)= SR∪ST	S\(R∩T)= S\R∩S\T	S(S\(SR)) =SR	S\(S(S\T)) =S\T
R∩ ⊣ R⇒	(A⇸B) → (A⇸B)	X↦R⇒(X∩R)	X⊑R⇒(X∩R)	R∩(R⇒Y)⊑Y	R∩(X∪Y)= (R∩X)∪(R∩Y)	R⇒(X∩Y)= (R⇒X)∩(R⇒Y)	R∩(R⇒(X∩R)) =X∩R	R⇒(R∩(R⇒Y)) =R⇒Y
𝓓 ⊣ ·⊤	(A⇸B) → Cor A	R↦(𝓓R)⊤	R⊑(𝓓R)⊤	𝓓(A⊤)⊑A	𝓓(R∪S)= 𝓓R∪𝓓S	(A∩B)⊤= A⊤∩B⊤	𝓓((𝓓R)⊤) =𝓓R	(𝓓(A⊤))⊤ =A⊤
𝓡 ⊣ ⊤·	(A⇸B) → Cor B	R↦⊤(𝓡R)	R⊑⊤(𝓡R)	𝓡(⊤A)⊑A	𝓡(R∪S)= 𝓡R∪𝓡S	⊤(A∩B)= ⊤A∩⊤B	𝓡(⊤(𝓡R)) =𝓡R	⊤(𝓡(⊤A)) =⊤A
·f ⊣ ·f°	(A⇸B) → (A⇸C)	ff°	𝟙⊑ff°	f°f⊑𝟙	(R∪S)f= Rf∪Sf	(R∩S)f°= Rf°∩Sf°	ff°f=f	f°ff°=f°
f°· ⊣ f·	(A⇸C) → (B⇸C)	ff°	𝟙⊑ff°	f°f⊑𝟙	f°(X∪Y)= f°X∪f°Y	f(X∩Y)= fX∩fY	ff°f=f	f°ff°=f°
i ⊣ E	Map ↪ Rel	E	$\frac{1}{\ni}$:A⇸E A	∋:E B⇸B	—	—	$\frac{\ni}{\ni}$=𝟙	$\frac{R}{\ni}$∋=R
·∋ ⊣ $\frac{}{\ni}$	Map(A,E B) → (A⇸B)	𝟙	$\frac{f\ni}{\ni}$=f	$\frac{R}{\ni}$∋=R	—	—	$\frac{f\ni}{\ni}$∋ =f∋	$\frac{\frac{R}{\ni}\ni}{\ni}$ =$\frac{R}{\ni}$
$\frac{}{\ni}$ ⊣ ·∋	(A⇸B) → Map(A,E B)	𝟙	$\frac{R}{\ni}$∋=R	$\frac{f\ni}{\ni}$=f	—	—	$\frac{\frac{R}{\ni}\ni}{\ni}$ =$\frac{R}{\ni}$	$\frac{f\ni}{\ni}$∋ =f∋

§1.2. Composing adjunctions

(1.2a)

$$\boxed{\frac{X}{f(R/T)}}_{\text{f a map}} \overset{}{\underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow}} \boxed{\frac{f^\circ\ X}{R/T}} \underset{\cdot T\dashv /T}{\Longleftrightarrow} \boxed{\frac{(f^\circ\ X)\ T}{R}}$$

$$\underset{\text{associativity}}{\Longleftrightarrow} \boxed{\frac{f^\circ(X\ T)}{R}} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \boxed{\frac{X\ T}{f\ R}} \underset{\cdot T\dashv /T}{\Longleftrightarrow} \boxed{\frac{X}{(f\ R)/T}}$$

$$\underset{\text{indirect equality}}{\Longrightarrow} \boxed{\frac{f(R/T)}{(f\ R)/T}}$$

(1.2b)

	·T	T·	·g	g°·	T∩	°	⟜◁	Δ
·S	R/(ST)=(R/T)/S	T\(R/S)=(T\R)/S	R/(Sg)=(Rg°)/S	g(R/S)=(gR)/S	—	(Y/S)°=S°\(Y°)	(RS)^=R^(S⊗𝟙)	$(T_1∩T_2)/S=T_1/S∩T_2/S$
S·	S\(R/T)=(S\R)/T	(TS)\R=S\(T\R)	S\(Rg°)=(S\R)g°	(g°S)\R=S\(gR)	—	(S\Y)°=Y°/S°	((S⊗𝟙)R)^=S R^	$S\backslash(T_1∩T_2)=S\backslash T_1∩S\backslash T_2$
·f	R/(fT)=(R/T)f°	T\(Rf°)=(T\R)f°	(fg)°=g°f°	—	—	(fY)°=Y°f°	—	$(T_1∩T_2)f°=T_1f°∩T_2f°$
f°·	f(R/T)=(fR)/T	(Tf°)\R=f(T\R)	—	(fg)°=g°f°	—	(fY)°=Y°f°	—	$f(T_1∩T_2)=fT_1∩fT_2$
R∩	—	—	—	—	(R∩T)⇒Y=R⇒(T⇒Y)	(R⇒Y)°=R°⇒(Y°)	—	$R⇒(T_1∩T_2)=(R⇒T_1)∩(R⇒T_2)$
°	(Y/T)°=T°\(Y°)	(T\Y)°=Y°/T°	(gY)°=Y°g°	(gY)°=Y°g°	(T⇒Y)°=T°⇒(Y°)	Y°°=Y	(R^)°=(R°⊗𝟙)(𝟙⊗▷⊸) both bends: the ° section's display	$(T_1∩T_2)°=T_1°∩T_2°$
⟜◁	(RT)^=R^(T⊗𝟙)	((T⊗𝟙)R)^=T R^	—	—	—	(R^)°=(R°⊗𝟙)(𝟙⊗▷⊸) both bends: the ° section's display	—	—
∪	$(X_1∪X_2)T=X_1T∪X_2T$	$T(X_1∪X_2)=TX_1∪TX_2$	$(X_1∪X_2)g=X_1g∪X_2g$	$g°(X_1∪X_2)=g°X_1∪g°X_2$	$T∩(X_1∪X_2)=(T∩X_1)∪(T∩X_2)$	$(X_1∪X_2)°=X_1°∪X_2°$	—	—
⊥	⊥T=⊥	T⊥=⊥	⊥g=⊥	g°⊥=⊥	T∩⊥=⊥	⊥°=⊥	—	—

§1.3. Adjoint triples

Beyond ∪ ⊣ Δ ⊣ ∩, which §1.1 already carries as two rows, the table holds one adjoint triple with content: ∃_f ⊣ f* ⊣ ∀_f along a map f, once on each side of the composite. f : A → B throughout this subsection.

(1.3a)

·f ⊣ ·f° ⊣ /f°
f°· ⊣ f· ⊣ f\

The first two links of each chain are the map shunting rules, and the third is §1.1′s division row read at a chosen divisor: ·S ⊣ /S at S := f°, and S· ⊣ S\ at S := f.

The third link is not the first over again. For a map f, /f collapses to ·f°; being a map is exactly what that collapse spends R/(f S) = (R/S) f°. — f° is not a map, so /f° does not collapse in turn, and the three operators stay distinct. On Rel(𝕀, A) = E A they are the image triple:

(1.3b)

operator	acts by	name
·f	S ↦ f[S]	direct image, ∃_f
·f°	T ↦ f^{-1}[T]	inverse image, f*
/f°	S ↦ {b : f^{-1}(b) ⊆ S}	∀_f

§1.1′s 𝓓 ⊣ ·⊤ is this same chain along the projection A⊗B → A, read through Rel(A, B) = E(A⊗B): 𝓓R = {(a,a) : ∃b. aRb}, A⊤ is the relation that ignores b altogether, and the third link is R ↦ 𝟙 ∩ R/⊤ = {(a,a) : ∀b. aRb}.

(1.3c)

𝓓 ⊣ ·⊤ ⊣ 𝟙∩·/⊤

Three is where it stops, and one map breaks both ends. Take A = $\{a_1,a_2\}$ and B = {b}: $f[\{a_1\}] ∩ f[\{a_2\}] = \{b\}$ while $f[\{a_1\} ∩ \{a_2\}] = ∅$, so ·f does not preserve meets and has no left adjoint, and the same two subsets give $∀\_f(\{a_1\} ∪ \{a_2\}) = \{b\}$ against $∀\_f\{a_1\} ∪ ∀\_f\{a_2\} = ∅$, so /f° does not preserve joins and has no right adjoint. A monic f restores the binary case and no more — ⊤f still reaches only im f, and ∀_f⊥ = B ∖ im f is still not ⊥. The chain extends only when f° is a map as well, and then /f° = ·f°° = ·f and it repeats forever. For an S with neither S nor S° a map there is no triple at all: ·S ⊣ /S is two links and stops.

The rows that chain forever say nothing by it: ° is an order-isomorphism of hom-posets, so it is adjoint to itself on both sides and ° ⊣ ° ⊣ ° has no end, and §1.1′s other row of that kind, ⟜◁ ⊣ ▷⊸, goes the same way.

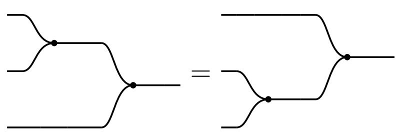
§2. Relations

(2a)

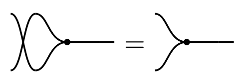
Rel is a poset-enriched category with (⊗, °) where ⊗ is commutative cartisian product, and converse ° ⊣ °, and

C: (▷ : A ⊗ A → A, ⟜ : 𝕀 → A) a commutative monoid with C° ⊣ C. We write C° as (◁ : A → A ⊗ A, ⊸ : A → 𝕀)

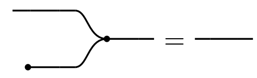
(2b)



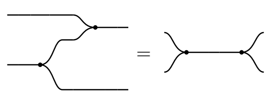
▷ associative



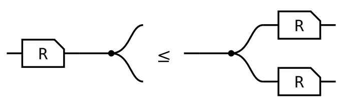
▷ commutative



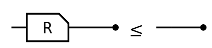
⟜ is its unit



Frobenius, one half — the other is its °



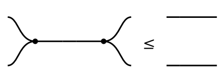
R◁ ≤ ◁(R⊗R)



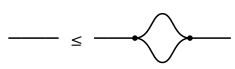
R⊸ ≤ ⊸

§2.1. ∀ object A, (A, ◁, ⊸) ⊣ (A, ▷, ⟜)

(2.1a)



▷ ◁ ≤ 𝟙



𝟙 ≤ ◁ ▷

⟜ ⊸ ≤ 𝟙

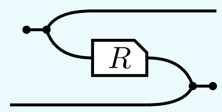
𝟙 ≤ ⊸ ⟜

The last of these is the only one that makes a picture **bigger**, and it is worth a name: **a wire is below the cut wire**. In `Rel` it reads `{(a, a)} ⊆ A × A` — cut a wire and its two ends stop having to agree, so cutting can only add pairs. It is the one weakening this calculus gives away for free.

§3. ° : $\mathcal{C}^{\mathsf{op}} \longrightarrow \mathcal{C}$ is a 2 functor

(3a)

° is primitive, part of the data the first section lists. It turns both of `R`'s wires round, and the Frobenius structure DRAWS that — the picture and the formula below are `R°`, not its definition:



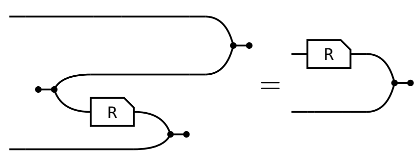
`R° = (⟜◁ ⊗ 𝟙) (𝟙 ⊗ R ⊗ 𝟙) (𝟙 ⊗ ▷⊸)`

where `⟜◁ : 𝕀 → a ⊗ a` opens a pair of wires out of nothing and `▷⊸ : b ⊗ b → 𝕀` closes one, so the input of `R°` is where the output of `R` was. Taking that formula as the definition would now be circular: `◁` and `⊸` are `▷°` and `⟜°`. The laws of ° are that it is a contravariant 2-functor `° : 𝒞ᵒᵖ → 𝒞`:

(i) `𝟙° = 𝟙` (ii) `(R S)° = S° R°` (iii) `(R ⊗ S)° = R° ⊗ S°` (iv) `R ≤ S` implies `R° ≤ S°`

§3.1. The slide

The one rule (ii) and (iii) use, and each of them uses it twice. A converse facing a merge on the lower strand is the box itself, upright, on the upper one:



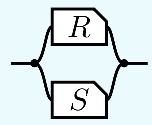
(3.1a)

`R°` is DEFINED as the bending of `(R ⊗ 𝟙) ▷⊸`, so the slide claims only that unbending it gives that back — and unbending undoes bending for every arrow. That is the snake above with a passenger: the `⟜◁` bends the `a` strand down and the `▷⊸` brings it back up, while the `b` strand rides through untouched. Nothing else is spent below.

§4. ∩ is a commutative idempotent monoid on every hom-set

(4a)

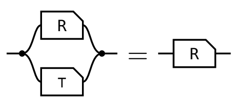
The **meet** of $R, S : a \to b$, the paper's **convolution**, is $R \cap S := \lhd (R \otimes S) \rhd$ — copy the input, run R and S on the two copies, merge the results — so what comes out is what both of them do.



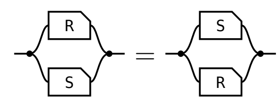
transcribed: a definition has no statement to export

On every hom-set it is associative, commutative and idempotent, with unit the maximal arrow $\top = ⊸\ ⟜$ (the paper's Lemma 4.11).

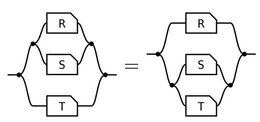
(4b)



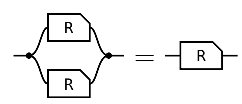
unit: one half of $\top$ per end — the merge's unit law absorbs the ⟜, the copy's counit law the ⊸



commutative: σ crosses $R \otimes S$ by naturality and is absorbed by cocommutativity and commutativity



associative: coassociativity and associativity; $\otimes$ re-brackets for nothing, being strict here

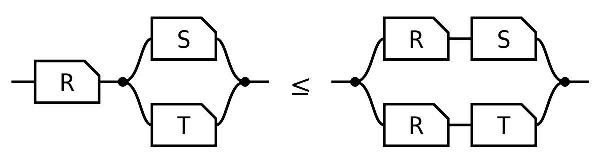


idempotent: the one that is not bookkeeping — the lax copy law is the whole of it, worked in allegory2

So $\le$ is the order this monoid induces. $R \cap S \le R$ comes from the unit, and idempotency turns anything under both S and T into something under $S \cap T$, since $R = R \cap R \le S \cap T$.

And one law relating ∩ to composition, which is **not** an equation:

(4c)

semi-distributivity, and what supplies it	**picture**
$R\ (S \cap T) \sqsubseteq R\ S \cap R\ T$ — the lax copy law. Equality exactly when R is single valued: the Maps section's $F\ (R \cap S) = F\ R \cap F\ S$.	

§5. Domain and range

> The **domain** Dom R ≜ 𝟙 ∩ R R° and the **range** Ran R ≜ Dom R°. (5a)

(5b)

▷ lands the return leg back on the value ◁ handed out,
so R° ▷ cuts to ⊸ — Frobenius

Running R and throwing the result away leaves only the fact that R could fire, and Ran R is the same picture with the box mirrored. In Rel both steps are {(a,a) : ∃b. a R b}.

(5c)

Dom R ≜ 𝟙 ∩ R R°
Dom R ⊑ A ⟺ R ⊑ A R, for A coreflexive
Dom (R S) ⊑ Dom R
Dom (R ∩ S) = 𝟙 ∩ S R°
R entire ⟺ Dom R = 𝟙 ⟺ 𝟙 ⊑ R R°
R simple ⟺ R° R ⊑ 𝟙
R a map ⟺ R entire and simple
R, S entire ⟹ R S entire — likewise simple, likewise maps
R S entire ⟹ R entire

§5.1. Sliding the discard

Dom (R S) ⊑ Dom R, and a single glyph for Dom would have nothing to slide: with the box and the discard drawn apart, the law is one dot walking back along the lower strand.

(5.1a)

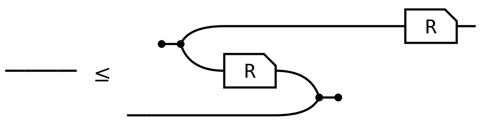
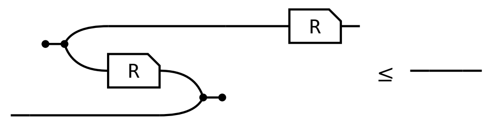
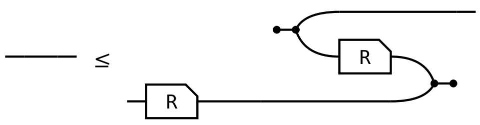
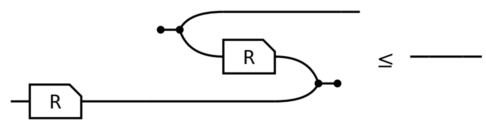
S ⊸ ⊑ ⊸, the lax axiom for ⊸ in the first section
— the discard slides back past S

Equality is S entire, which is the same picture read as Dom R = 𝟙 ⟺ R entire.

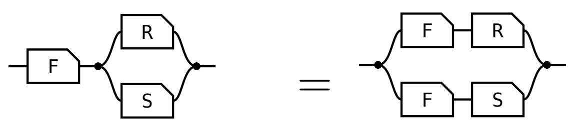
§6. Maps

Every arrow is lax for ◁ and for ⊸ by axiom, and for ▷ and ⟜ by taking $^\circ$ of those two. All four hold for **every** arrow, their REVERSES do not, and each reverse holding is a property of the arrow. The right-hand column is the **adjoint** form, which is the definition used here because it is literally the allegory's `Simple` and `Entire`; that the two forms agree is a separate theorem, not proved here.

(6a)

 **surjective** $\mathbb{1} \sqsubseteq R^\circ\ R$, that is, R° entire.	 **single valued** $R^\circ\ R \sqsubseteq \mathbb{1}$
 **entire** $\mathbb{1} \sqsubseteq R\ R^\circ$. With **single valued**, a **map**.	 **injective** $R\ R^\circ \sqsubseteq \mathbb{1}$

(6b)

law	picture
$F\ (R \cap S) = F\ R \cap F\ S$ F single valued F = people x may ask, R = admires a, S = admires b. Single valued means one person, so the sides agree.	 one person who admires both a at A, b at B

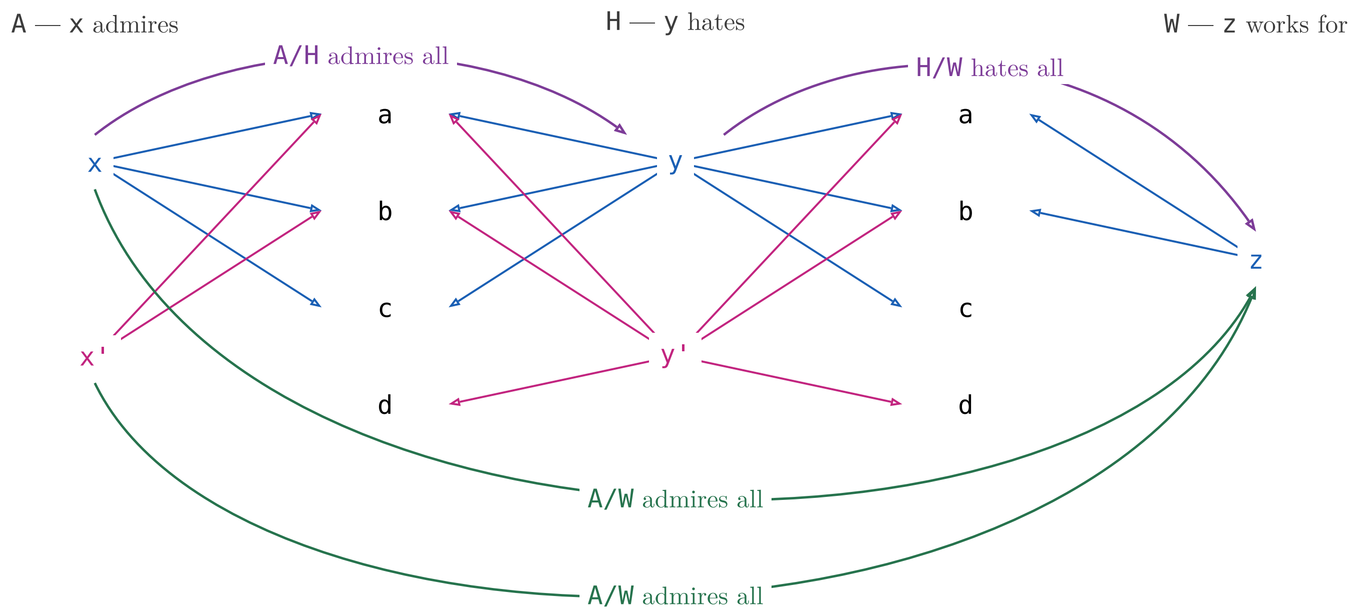
§7. `/` is all of

(7a)

```
x (R/S) y   ⟺   ∀p. y S p → x R p
x (S\R) y   ⟺   ∀p. p S x → p R y
/ compares images: S(y) ⊆ R(x).   \ compares preimages: S°(x) ⊆ R°(y).
example: A admires, H hates, W works for
x (A/H) y — x admires everyone y hates.
x (H\A) y — everyone who hates x admires y.
```

§7.1. `(R/S)(S/W) ⊑ R/W`

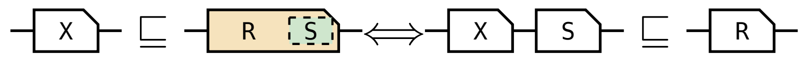
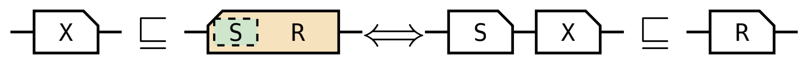
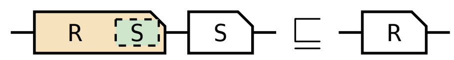
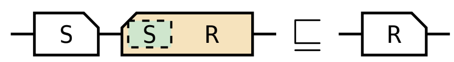
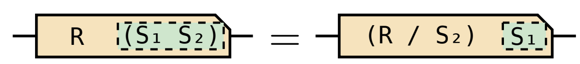
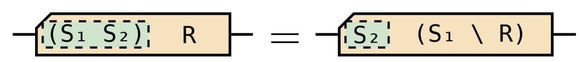
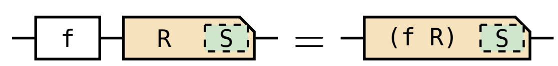
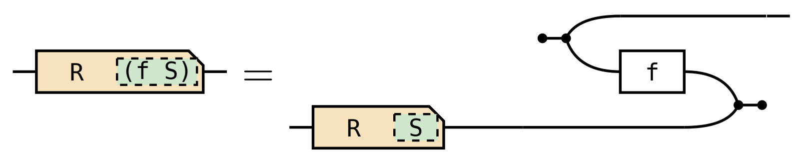
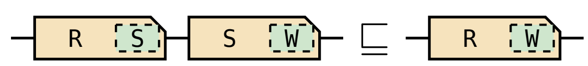
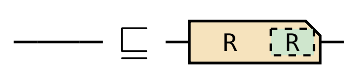
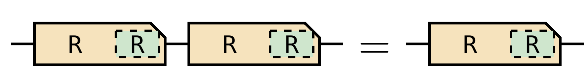
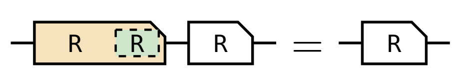
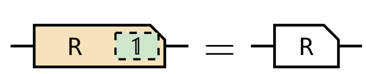
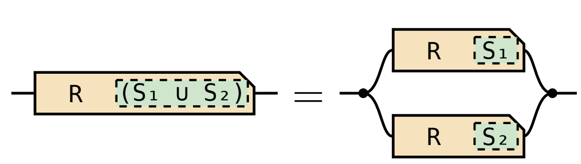
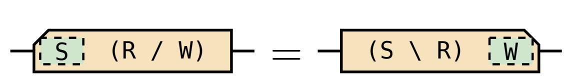
(7.1a)



`x (A/H) y` — `x` admires everyone `y` hates (7.1b)
`y (H/W) z` — `y` hates everyone `z` works for
`x (A/W) z` — `x` admires everyone `z` works for

`(A/H)(H/W)` is a path: `x` → `y` → `z`, and that is all of it. `A/W` also holds of `x'`, who admires everyone `z` works for — but `x'` does not admire everyone anybody hates, so nothing composes to it. The missing path is exactly the strictness of `(R/S)(S/W) ⊑ R/W`.

(7.1c)

$X \sqsubseteq R/S \iff X\ S \sqsubseteq R$ X is any x-to-y pairing; one that only pairs x with a y such that x admires everyone y hates lies inside R/S, and R/S is the largest such.	
$X \sqsubseteq S\backslash R \iff S\ X \sqsubseteq R$ The mirror — divide on the left when X comes first.	
$(R/S)\ S \sqsubseteq R$ There is a y such that x admires everyone y hates, and p is one of the people y hates — then x admires p too. Strict at $S = \varnothing$: R/S is everyone, $(R/S)\ S = \varnothing$.	
$S\ (S\backslash R) \sqsubseteq R$ The mirror.	
associate: $R/(S_1\ S_2) = (R/S_2)/S_1$ **A friend's enemy** is two hops: divide by the far end first.	
$(S_1\ S_2)\backslash R = S_2\backslash(S_1\backslash R)$ The mirror.	
maps: $f\ (R/S) = (f\ R)/S$ Rename x before or after dividing — the licence to write $f\ R\ /\ S$.	
$R/(f\ S) = (R/S)\ f°$ Rename y: a map leaves a denominator as $f°$ outside the box.	
$(R/S)(S/W) \sqsubseteq R/W$ Someone who admires all of a hate-set that already covers everyone z works for admires those people too.	
$𝟙 \sqsubseteq R/R$ R/R runs admirer to admirer: each admires everyone they admire. Strict: two people who each admire only a and b admire each other's idols too, and still stay two people.	
$(R/R)(R/R) = R/R$ R/R is the preorder **admires at least as much as**, and a preorder is idempotent. Freyd writes $\sqsubseteq$; with $𝟙 \sqsubseteq R/R$ above it is an equality.	
$(R/R)\ R = R$ Reaching p through someone whose idols x fully admires is reaching p directly, since x admires their own idols.	
$R/𝟙 = R$ Dividing by 𝟙: p's set is just $\{p\}$, so admiring all of it is admiring p.	
$R/(S_1 \cup S_2) = R/S_1 \cap R/S_2$ Admiring a combined hate-set is admiring each set in full.	
$S\backslash(R/W) = (S\backslash R)/W$ Which is why $S\backslash R/W$ needs no bracket.	

Fifteen laws, fifteen pictures, and not one shows a generator: ∩, ∪, ° and composition are what the Frobenius generators build, and / is none of those — it is posited, with nothing to unfold.

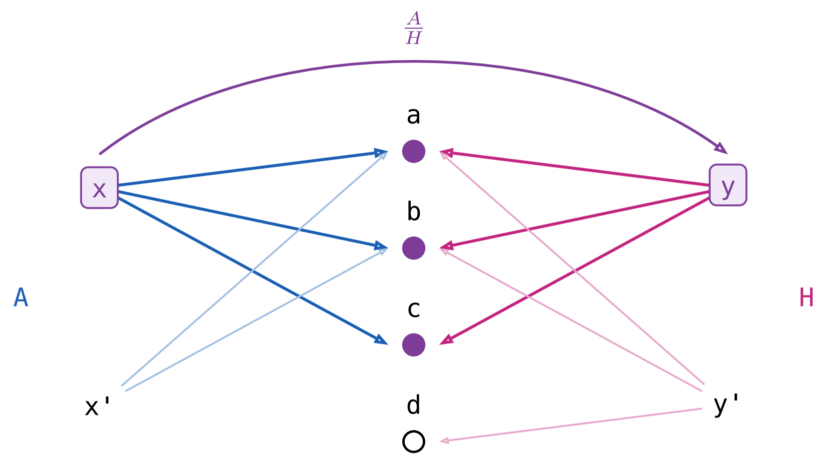
§8. $\frac{R}{S}$

> $\frac{R}{S}$ ≜ `(R/S) ∩ (S/R)°`. In `Rel` `x` and `y` has the same image: `∀p. (x R p ⟺ y S p)` (8a)

`x (A/H) y` — `x` admires everyone `y` hates (8b)
`x ((H/A)°) y` — `x` admires only people `y` hates
`x ((A/H) ∩ (H/A)°) y` — `x` admires only and all whom `y` hates

`x` admires exactly whom `y` hates: `/` is **all of**, $\frac{R}{S}$ is **only all of**.

(8c)



(8d)

law	the reading
$X \sqsubseteq \frac{R}{S} \iff XS \sqsubseteq R$ and `X° R ⊑ S`	`X` may pair `x` with `y` only when `x` admires exactly whom `y` hates. Both halves must typecheck, so the operation is **partial**.
$\left(\frac{R}{S}\right)^{\circ} = \frac{S}{R}$	Matching is symmetric.
$\frac{R}{S}\frac{S}{W} \sqsubseteq \frac{R}{W}$	And transitive.
$\frac{R}{S}S \sqsubseteq R$	$\left(\exists y.\, x\left(\frac{R}{S}\right)y \wedge ySp\right) \rightarrow xRp$ `x only admires whom y hates` $\frac{R}{S}S = \mathrm{Dom}\left(\frac{R}{S}\right)R$

$\frac{R}{R}R = R$	$\left(\exists y.\, x\left(\frac{R}{R}\right)y \wedge yRp\right) \iff xRp$
	`x and y admire the same people`
	`y = x always qualifies (𝟙 ⊑ R%R below)`

$\mathbb{1} \sqsubseteq \frac{R}{R}$	$x\left(\frac{R}{R}\right)y$ if `x` and `y` admires the same peoples.
$\left(\frac{R}{R}\right)^{2} = \frac{R}{R}$	So the relation **admires the same people** is an equivalence relation.
$X \sqsubseteq \frac{R}{R} \iff XR \sqsubseteq R$, for symmetric `X`	The largest symmetric arrow that leaves `R` alone.
$\frac{R}{\mathbb{1}}$ is the **simple part** of `R`	The people who admire exactly one person and nobody else. It equals `R` only when `R` is simple, unlike `R/𝟙 = R`.
`Dom` $\frac{R}{S}$ `= 𝟙 ∩ (R/S)(S/R)`	Its domain is the **domain of simplicity** of `R`.

§9. Freyd's Power allegories

(9a)

A **power allegory** is a division allegory with one unary operation on arrows, ∋ **epsiloff**, subject to

$$\begin{array}{ll} \ni_{\mathrm{R}}\Box = R\Box, \quad \ni_{\mathrm{R}} = \ni_{\mathrm{R}\,\Box} & \\ \mathbb{1} \sqsubseteq (R/\ni_{\mathrm{R}})(\ni_{\mathrm{R}}/R) & \ni_{\mathrm{R}} \text{ is } \textbf{thick} \\ \frac{\ni_{\mathrm{R}}}{\ni_{\mathrm{R}}} = \mathbb{1} & \ni_{\mathrm{R}} \text{ is } \textbf{straight} \end{array}$$

`R □` is `R`'s target, an identity arrow. For `R : A → B` write `∋ : E B → B`, dropping the subscript.

`∈ ≜ ∋° : A → E A`

(9b)

	law	the reading
1	$\ni_{\mathrm{R}}\Box = R\Box, \quad \ni_{\mathrm{R}} = \ni_{\mathrm{R}\,\Box}$	One ∋ per object, not per arrow.
2	∋ is **thick**	**Comprehension**: every `x` has a set of exactly the people `x` admires.
3	$\frac{R}{\ni}$ `: A → E B`, for `R : A → B`	convert a relation to a function. `a` $\frac{R}{\ni}$ `= {b｜a R b}`
4	$\frac{\mathsf{R}}{\ni}$ is a **map**	
5	$\frac{\mathsf{R}}{\ni}$ `∋ = R`	
6	`F ⊑` $\frac{\mathsf{F}\ \ni}{\ni}$, `F` simple	A partial choice of sets is inside the total one.
7	$\frac{\mathbb{1}}{\ni}$, the **singleton map**, monic	The one-person set.
8	$\frac{\ni}{\ni}$ `= 𝟙`	Make the set of a set, then read it back one level down.
9	**fusion:** $\frac{\mathsf{f}\ \mathsf{R}}{\ni}$ `= f` $\frac{\mathsf{R}}{\ni}$, `f` a map	Naturality of the unit, `f` $\frac{\mathbb{1}}{\ni}$ `=` $\frac{\mathbb{1}}{\ni}$ `(E(f))`.
10	$\frac{\mathsf{f}}{\ni}$ `= f` $\frac{\mathbb{1}}{\ni}$, `f` a map	Rename first or take singletons first — the fusion row above at `R = 𝟙`.
11	$\frac{R}{S} = \frac{R}{\ni}\left(\frac{S}{\ni}\right)^{\circ}$	`x` and `y` match when `R` sends `x` and `S` sends `y` to the same set.
12	`E(R) ≜` $\frac{\ni\ \mathsf{R}}{\ni}$, `E ≜` $\frac{\ni\cdot}{\ni}$	`E(R): E A → E B`, image of a set of A
13	$\frac{\mathsf{R}}{\ni}$ `=` $\frac{\mathbb{1}}{\ni}$ `E(R)`	$\frac{\mathbb{1}}{\ni}$`: x ↦ {x}`
14	$\frac{\mathsf{S}}{\ni}$ `E(R) =` $\frac{\mathsf{S}}{\ni}$ $\frac{\ni\ \mathsf{R}}{\ni}$ `=` $\frac{\mathsf{S}\ \mathsf{R}}{\ni}$	absorption
15	`subset ≜ ∋/∋ : E A → E A`	`xs subset ys ⟺ ∀a. ys ∋ a → xs ∋ a`, that is `ys ⊆ xs`, not `xs ⊆ ys`.

§9.1. `i ⊣ E` Power Allegory defined as adjunction

(9.1a)

$$\begin{tikzcd} \mathsf{A} \arrow[r, "\mathsf{R}"] \arrow[d, "\frac{\mathbb{1}}{\ni}"'] \arrow[dr, dashed, "\frac{\mathsf{R}}{\ni}"] & \mathsf{B} \\ \mathsf{E\ A} \arrow[r, "{\mathsf{E(R)}}"', "{\mathsf{E} = \frac{\ni\cdot}{\ni}}"' yshift=-12pt] & \mathsf{E\ B} \arrow[u, "\ni"'] \end{tikzcd}$$

$\frac{\mathsf{R}}{\ni}$ `∋ = R`

`E A` is the powerset of `A`. B&dM write `P A` — standard mathematics, but here `P` is already the relator `P(R)`.

§10. Relator

(10a)

Every hom-set of an allegory is a poset, so an allegory is a **locally posetal 2-category**: the 2-cell from `R` to `S` IS `R ⊑ S`. A **relator** `F : 𝒞 → 𝓓` is a 2-functor between allegories:

$$F(\mathbb{1}) = \mathbb{1} \qquad F(R\ S) = F(R)\ F(S) \qquad R \sqsubseteq S \implies F(R) \sqsubseteq F(S)$$

Preserving $°$ is **not** asked for — $°$ is an identity-on-objects involution $\mathcal{C}^{op} \to \mathcal{C}$, no part of the 2-category.

(10b)

the statement	**B&dM**
For f a map, $F(f)$ is a map and $F(f°) = F(f)°$.	Lemma 5.1
Over a **tabular** allegory a functor is a relator $\iff$ it preserves $°$.	Theorem 5.1
$F(R°) = F(R)°$ for every R, so $F(R)°$ needs no bracket.	after Theorem 5.1, p. 113
Two relators agreeing on maps are equal.	Corollary 5.1
$F(X \cap Y) = F(X) \cap F(Y)$ for $X,\ Y$ coreflexive.	Ex 5.2
$F(R \cap S) \sqsubseteq F(R) \cap F(S)$, and strictly.	Ex 5.2, the restriction
$F(\mathrm{Dom}\ R) = \mathrm{Dom}(F(R))$ for F preserving $°$.	—

The **power relator** P — $xs\ P(R)\ ys \iff (\forall a \in xs.\ \exists b \in ys.\ a\ R\ b) \wedge (\forall b \in ys.\ \exists a \in xs.\ a\ R\ b)$ — is where the fourth is strict: for $R = \{(a_1,b_1),\ (a_2,b_2)\}$ and $S = \{(a_1,b_2),\ (a_2,b_1)\}$ the pair $(\{a_1,a_2\},\ \{b_1,b_2\})$ is in $P(R) \cap P(S)$, while $R \cap S = \varnothing$.

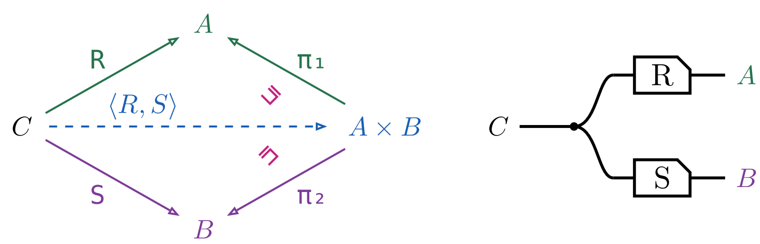
§11. Fork $\langle R,S\rangle$

(11a)

The **fork** of $R\ :\ C \to A$ and $S\ :\ C \to B$ is $\langle R,S\rangle \triangleq R\ \pi_1° \cap S\ \pi_2°$, where $(\pi_1,\ \pi_2)$ is the tabulation of $\top$.

$$\langle R,S\rangle\ \pi_1 = (\mathrm{Dom}\ S)\ R \qquad \langle R,S\rangle\ \pi_2 = (\mathrm{Dom}\ R)\ S \tag{11b}$$

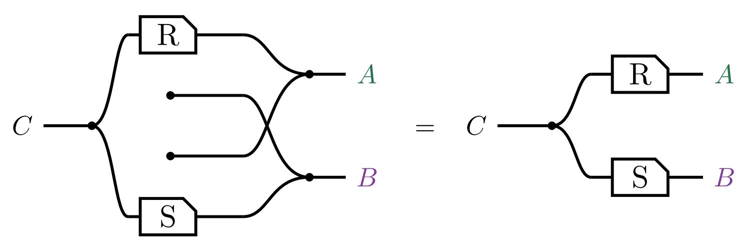
(11c)



A domain is coreflexive, so $\langle R,S\rangle\ \pi_1 \sqsubseteq R$, with equality exactly when S is entire; for maps both triangles commute and $\langle f,g\rangle$ is unique. In Rel, $c\ \langle R,S\rangle\ (a,b)$ iff $c\ R\ a$ and $c\ S\ b$ — copy c, then R on one strand and S on the other, which is $\lhd\ (R \otimes S)$ on the right.

No $°$ survives the translation. $\pi_1 = \mathbb{1} \otimes \multimap$ discards the second component, so $\pi_1° = \mathbb{1} \otimes ⟜$ **creates** one out of nothing, and $\cap$ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:

(11d)



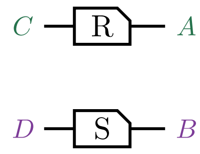
$⟜\ \rhd = \mathbb{1}$ on each half

§11.1. Relational product $R \times S$

(11.1a)

$R \times S \triangleq \langle \pi_1\ R,\ \pi_2\ S\rangle$, a relator in each argument but no longer a categorical product.

$$\begin{tikzcd}
C \arrow[rr, "\mathsf{R}"] & & A \\
C \times D \arrow[u, "\pi_1"] \arrow[d, "\pi_2"'] \arrow[rr, dashed, "R \times S"] \arrow[urr, phantom, "\sqsubseteq"] \arrow[drr, phantom, "\sqsubseteq"] & & A \times B \arrow[u, "\pi_1"'] \arrow[d, "\pi_2"] \\
D \arrow[rr, "\mathsf{S}"] & & B
\end{tikzcd}$$

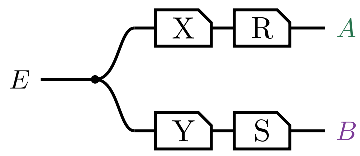


(11.1b)

Right-then-up is $(R \times S)\ \pi_1$, up-then-right is $\pi_1\ R$, and $(R \times S)\ \pi_1 \sqsubseteq \pi_1\ R$, equality when S is entire. In Rel, $(c,d)\ (R \times S)\ (a,b)$ iff $c\ R\ a$ and $d\ S\ b$ — two strands side by side, no copy dot: $R \times S = R \otimes S$.

§11.2. Absorption

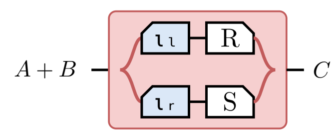
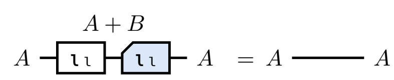
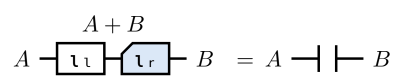
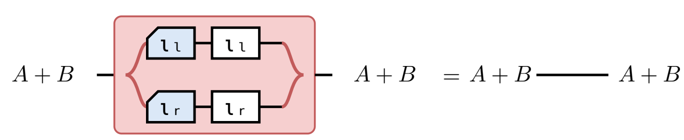
For $X : E \to C$ and $Y : E \to D$, $\langle X,Y\rangle\ (R \times S) = \langle X\ R,\ Y\ S\rangle$. Both sides are this picture:



(11.2a)

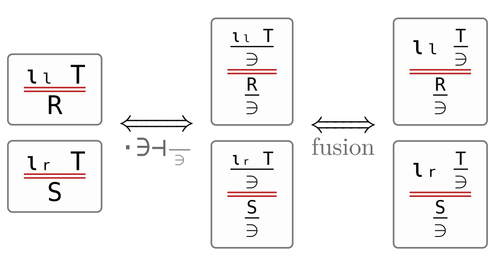
§12. Coproduct $[R,S]$

(12a)

the statement	picture
$[R,S] \triangleq \iota_l{}^\circ\ R \cup \iota_r{}^\circ\ S$ The tape is the union — a particle entering at $A + B$ takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.	
$[R,S] = [\frac{R}{\ni}, \frac{S}{\ni}]\ \ni$	
$R + S \triangleq [R\ \iota_l,\ S\ \iota_r]$	
$\iota_l\ [R,S] = R$, $\iota_r\ [R,S] = S$, and $[R,S]$ is the only such arrow	
$\iota_l\ \iota_l{}^\circ = \mathbb{1} = \iota_r\ \iota_r{}^\circ$	
$\iota_l\ \iota_r{}^\circ = \bot = \iota_r\ \iota_l{}^\circ$	
$\iota_l{}^\circ\ \iota_l \cup \iota_r{}^\circ\ \iota_r = \mathbb{1}$	
$[U,V]^\circ\ [R,S] = U^\circ\ R \cup V^\circ\ S$	

§12.1. $[R,S] \triangleq [\frac{R}{\ni}, \frac{S}{\ni}]\ \ni$

The universal-property row is not free: $\iota_l,\ \iota_r$ were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because $\frac{}{\ni}$ sends an arrow $A \to C$ to a map $A \to E\ C$ reversibly, so the map coproduct can be applied underneath it. For any $T : A + B \to C$,



(12.1a)

$$\xLeftrightarrow[\text{coproduct of maps}]{} \boxed{\dfrac{\frac{\mathsf{T}}{\ni}}{[\frac{\mathsf{R}}{\ni},\ \frac{\mathsf{S}}{\ni}]}} \xLeftrightarrow[\cdot\ni\dashv\frac{}{\ni}]{} \boxed{\dfrac{\mathsf{T}}{[\frac{\mathsf{R}}{\ni},\ \frac{\mathsf{S}}{\ni}]\ \ni}}$$

$$\begin{tikzcd}
 & A \arrow[dl, "\iota_l"'] \arrow[dr, "\frac{\mathsf{R}}{\ni}"] \arrow[rr, bend left, "\mathsf{R}"] & & \\
A+B \arrow[rr, dashed, "{[\frac{\mathsf{R}}{\ni},\ \frac{\mathsf{S}}{\ni}]}"] & & EC \arrow[r, "\ni"] & C \\
 & B \arrow[ul, "\iota_r"] \arrow[ur, "\frac{\mathsf{S}}{\ni}"'] \arrow[urr, bend right, "\mathsf{S}"'] & &
\end{tikzcd} \tag{12.1b}$$

Nothing here holds only up to $\sqsubseteq$: every triangle commutes on the nose, which is the difference from the fork above. The border spells $[\mathsf{R},\mathsf{S}] = [\frac{\mathsf{R}}{\ni},\ \frac{\mathsf{S}}{\ni}]\ \ni$, and pushing $\ni$ into the union that $[\cdot,\cdot]$ on maps already is turns that back into the definition, $(\iota_l{}^\circ\ \frac{\mathsf{R}}{\ni} \cup \iota_r{}^\circ\ \frac{\mathsf{S}}{\ni})\ \ni = \iota_l{}^\circ\ \mathsf{R} \cup \iota_r{}^\circ\ \mathsf{S}$.

§13. The power relator P(R)

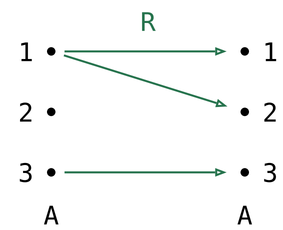
(13a)

For $R : A \to B$, $P(R) \triangleq ((\ni R)/\ni) \cap ((\ni R^{\circ})/\ni)^{\circ} : E\,A \to E\,B$.

$xs\ P(R)\ ys \iff (\forall a \in xs.\ \exists b \in ys.\ a\ R\ b) \wedge (\forall b \in ys.\ \exists a \in xs.\ a\ R\ b)$

Every element of xs is related by R to some element of ys, and conversely.

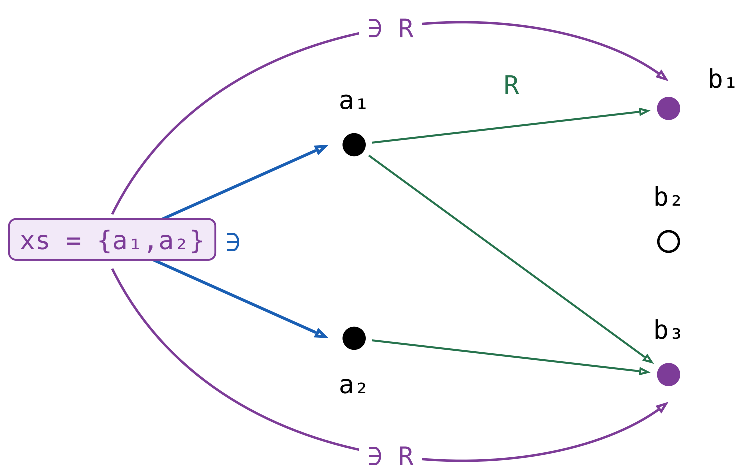
(13b)



xs	ys under (∋ R)/∋	ys under P(R)	ys under E(R)
∅	∅	∅	∅
{1}	∅, {1}, {2}, {1,2}	{1}, {2}, {1,2}	{1,2}
{2}	∅	none	∅
{3}	∅, {3}	{3}	{3}
{1,2}	∅, {1}, {2}, {1,2}	none	{1,2}
{1,3}	all 8 subsets of {1,2,3}	{1,3}, {2,3}, {1,2,3}	{1,2,3}
{2,3}	∅, {3}	none	{3}
{1,2,3}	all 8 subsets of {1,2,3}	none	{1,2,3}

R(2) = ∅, so P(R) sends no ys at all to any xs containing 2 — not entire.
R(1) has two elements, so {1} gets three: 1 need only meet ys, not be swallowed by it — not simple.
Every E(R) row has exactly one ys, which is what makes it a map.

(13c)

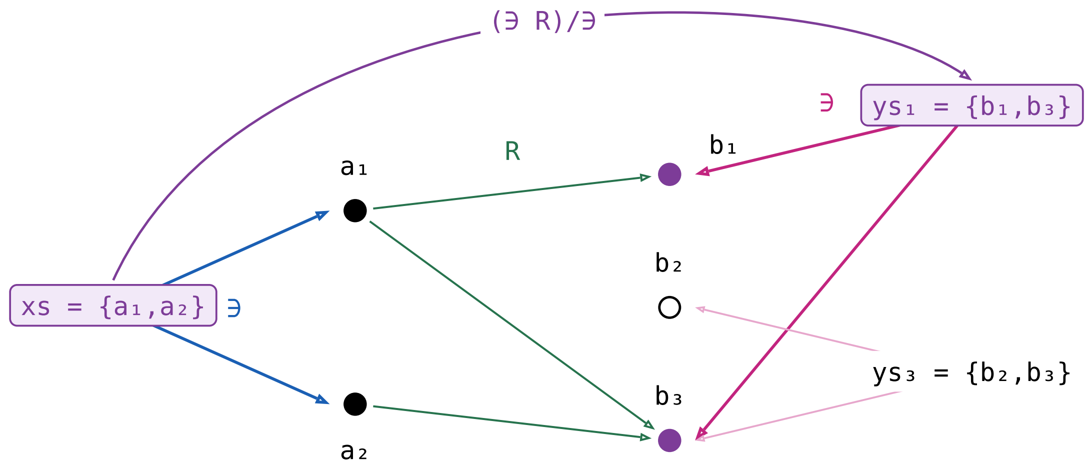


$\ni R : E\,A \to B \qquad xs\ (\ni R)\ b \iff \exists a \in xs.\ a\ R\ b$

one arc per element of B that xs reaches, and b_2 gets none

Dividing by ∋ turns that into a relation between **sets**.

(13d)

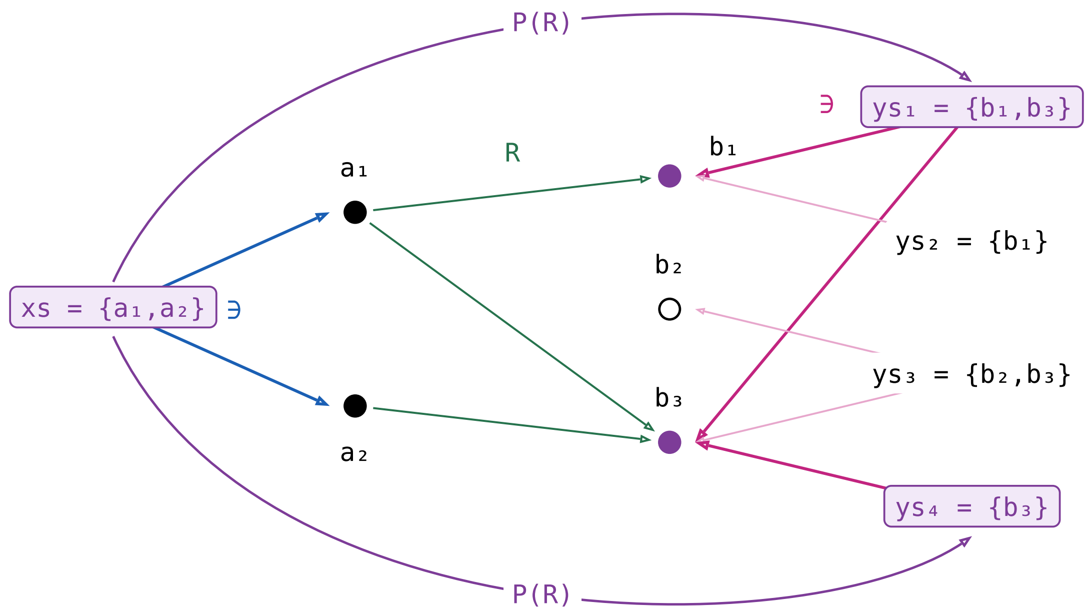


$$(\ni\ R)/\ni\ :\ E\ A \to E\ B \qquad xs\ ((\ni\ R)/\ni)\ ys \iff \forall b \in ys.\ \exists a \in xs.\ a\ R\ b$$

ys_3 is rejected: it names b_2, which xs does not reach

The other half of the meet is that clause with the sets swapped and R turned round: every element of xs must reach ys:

(13e)



$$((\ni\ R^\circ)/\ni)^\circ\ :\ E\ A \to E\ B \qquad xs\ ((\ni\ R^\circ)/\ni)^\circ\ ys \iff \forall a \in xs.\ \exists b \in ys.\ a\ R\ b$$

ys_2 is rejected too: a_2's only image is b_3, which ys_2 does not name

(13f)

law	**the reading**
$X \sqsubseteq P(R) \iff X\ \ni\ \sqsubseteq\ \ni\ R$ and $X^\circ\ \ni\ \sqsubseteq\ \ni\ R^\circ$	One containment, and the same one at R° — which is the definition read off the two divisions. Hence $P(R^\circ) = P(R)^\circ$, and $R \sqsubseteq S \implies P(R) \sqsubseteq P(S)$.
$P(\mathbb{1}) = \frac{\ni}{\ni} = \mathbb{1}$	The straightness axiom verbatim: extensionality **is** P's unit law.
$P(f) = \frac{\ni f}{\ni}$, for f a map	In Rel, $xs\ P(f)\ ys \iff ys = \{f\ a \mid a \in xs\}$. The half at f° says every $a \in xs$ has its $f\ a$ on ys; f has just the one image per a, so that already says ys contains everything xs reaches, which is the fraction's second half. For a map the two definitions coincide.
$P(R\ S) = P(R)\ P(S)$	$\sqsupseteq$ is the division cancellation laws. $\sqsubseteq$ is the one law in this section that is not a calculation: it needs a tabulation of $P(R\ S)$.

§14. Catamorphism

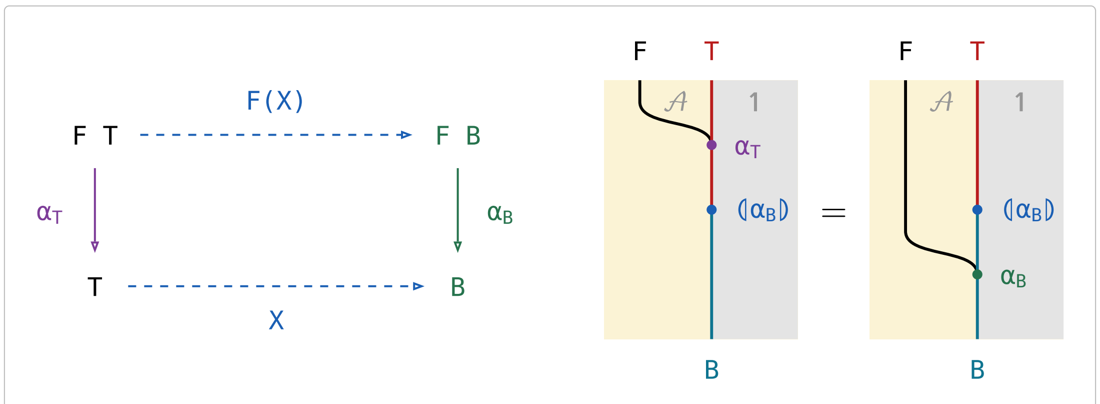
(14a)

> F a relator with an **initial algebra** $\alpha_\mathsf{T} : \mathsf{F\ T} \to \mathsf{T}$ among the maps. For a relational algebra $\alpha_\mathsf{B} : \mathsf{F\ B} \to \mathsf{B}$, the **catamorphism** $(\!|\alpha_\mathsf{B}|\!) : \mathsf{T} \to \mathsf{B}$ is the unique arrow with $\alpha_\mathsf{T}\ (\!|\alpha_\mathsf{B}|\!) = \mathsf{F}((\!|\alpha_\mathsf{B}|\!))\ \alpha_\mathsf{B}$. Explicitly, for a relation $\mathsf{R} : \mathsf{F\ B} \to \mathsf{B}$, $(\!|\mathsf{R}|\!) \triangleq (\!|\frac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}|\!)\ \ni$, the inner $(\!|\cdot|\!)$ being the catamorphism of a **map** algebra.

An F-algebra is a **weakened** natural transformation. A transformation $\mathsf{F} \Rightarrow \mathbb{1}$ on $\mathcal{A}$ would need a component $\mathsf{F\ X} \to \mathsf{X}$ at every object and a commuting square at every arrow, but F-Algebra only need it works on T and B.

§14.1. The defining equation

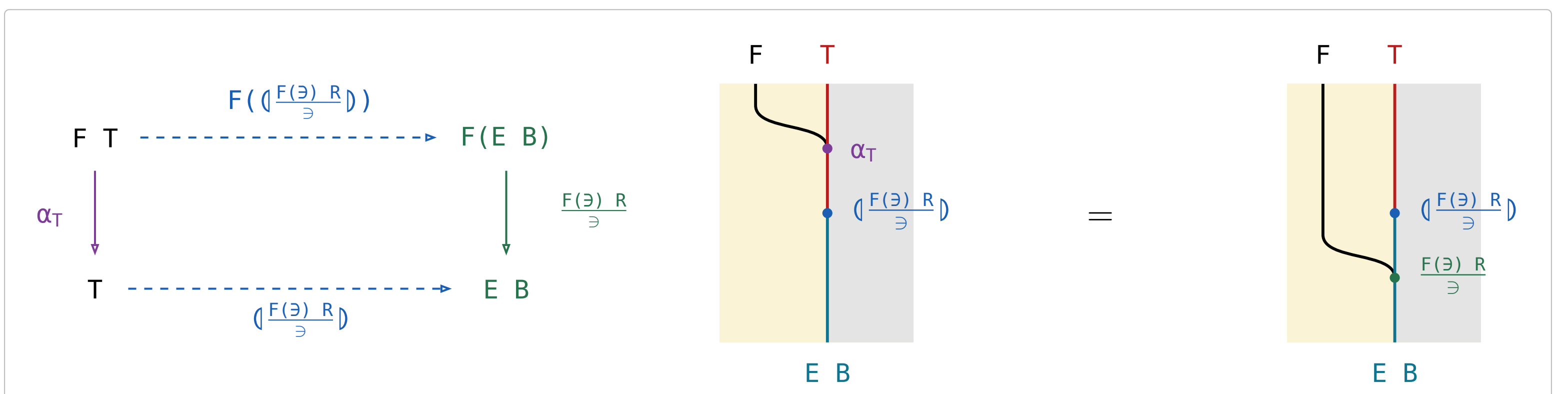
(14.1a)



$$\mathsf{X} = (\!|\alpha_\mathsf{B}|\!) \iff \alpha_\mathsf{T}\ \mathsf{X} = \mathsf{F(X)}\ \alpha_\mathsf{B}$$

§14.2. $(\!|\mathsf{R}|\!) = (\!|\frac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}|\!)\ \ni$

(14.2a)



$$\alpha_\mathsf{T}\ (\!|\tfrac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}|\!) = \mathsf{F}((\!|\tfrac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}|\!))\ \tfrac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}$$

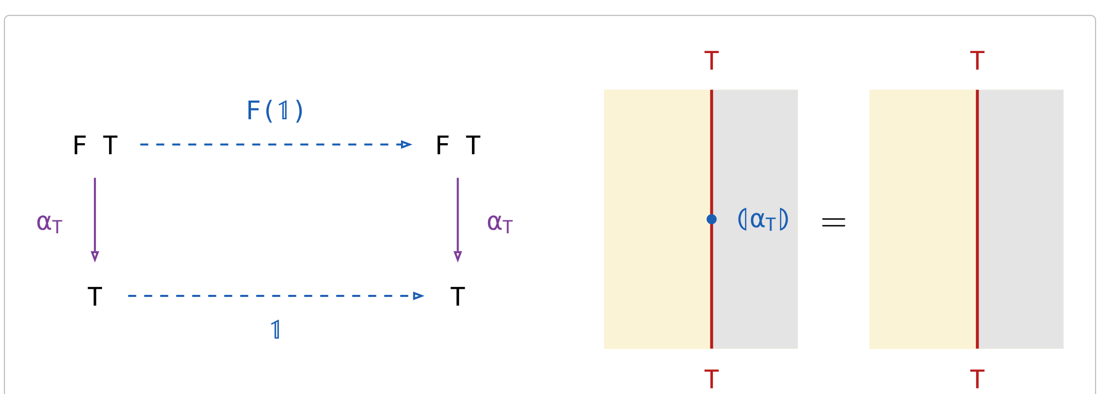
(14.2b)

$$\boxed{\frac{\alpha_\mathsf{T}\ \mathsf{X}}{\mathsf{F(X)\ R}}} \underset{\cdot\ni\dashv\frac{}{\ni}}{\Longleftrightarrow} \boxed{\frac{\frac{\alpha_\mathsf{T}\ \mathsf{X}}{\ni}}{\frac{\mathsf{F(X)\ R}}{\ni}}} \underset{\cdot\ni\dashv\frac{}{\ni}}{\Longleftrightarrow} \boxed{\frac{\frac{\alpha_\mathsf{T}\ \mathsf{X}}{\ni}}{\frac{\mathsf{F}(\frac{\mathsf{X}}{\ni}\ \ni)\ \mathsf{R}}{\ni}}}$$

$$\underset{\text{relator, fusion twice}}{\Longleftrightarrow} \boxed{\frac{\alpha_\mathsf{T}\ \frac{\mathsf{X}}{\ni}}{\mathsf{F}(\frac{\mathsf{X}}{\ni})\ \frac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}}} \underset{\text{catamorphism of maps}}{\Longleftrightarrow} \boxed{\frac{\frac{\mathsf{X}}{\ni}}{(\!|\frac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}|\!)}} \underset{\cdot\ni\dashv\frac{}{\ni}}{\Longleftrightarrow} \boxed{\frac{\mathsf{X}}{(\!|\frac{\mathsf{F}(\ni)\ \mathsf{R}}{\ni}|\!)\ \ni}}$$

§14.3. Reflection

(14.3a)



$$(\!|\alpha_\mathsf{T}|\!) = \mathbb{1}$$

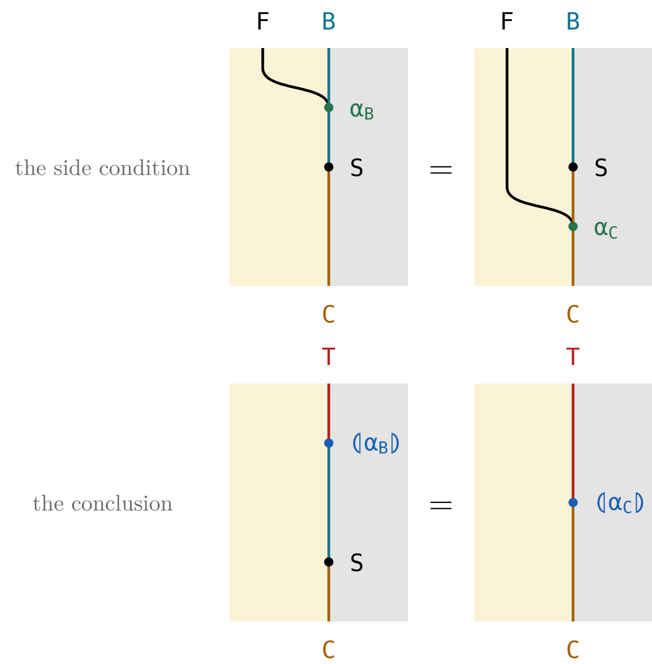
Taking a value apart with α_T and putting it straight back is doing nothing.

§14.4. Fusion

Fusion rewrites $\llparenthesis \alpha_B \rrparenthesis\ S$ through a second algebra $\alpha_C : F\ C \to C$ along an arrow $S : B \to C$.

(14.4a)

$$
\begin{tikzcd}
F\ T \arrow[r, dashed, "F(\llparenthesis \alpha_B \rrparenthesis)"] \arrow[d, "\alpha_T"'] & F\ B \arrow[r, "F(S)"] \arrow[d, "\alpha_B"] & F\ C \arrow[d, "\alpha_C"] \\
T \arrow[r, dashed, "\llparenthesis \alpha_B \rrparenthesis"'] & B \arrow[r, "S"'] & C
\end{tikzcd}
$$



$$\llparenthesis \alpha_B \rrparenthesis\ S = \llparenthesis \alpha_C \rrparenthesis \Longleftarrow \alpha_B\ S = F(S)\ \alpha_C$$

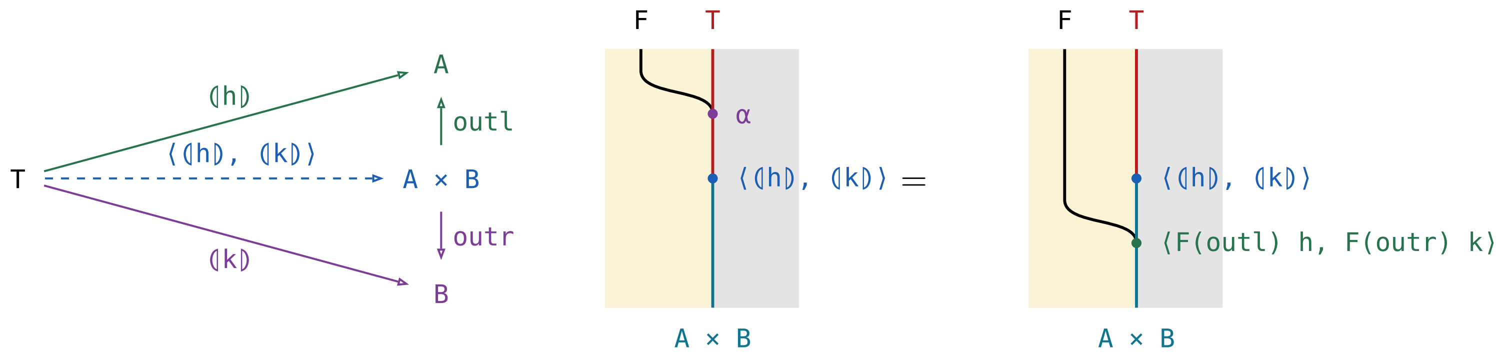
The left square is $\llparenthesis \alpha_B \rrparenthesis$'s own defining square and the right one is the side condition, so the outer rectangle says $\llparenthesis \alpha_B \rrparenthesis\ S$ satisfies the defining equation of $\llparenthesis \alpha_C \rrparenthesis$ — and uniqueness finishes it. A fold followed by `S` has collapsed into a single fold, which is how an intermediate structure is got rid of.

The side condition is (14.1a)'s picture again under $\alpha_T \mapsto \alpha_B,\ \llparenthesis \alpha_B \rrparenthesis \mapsto S,\ \alpha_B \mapsto \alpha_C$: both say that the bead between the two merges is a homomorphism of `F`-algebras.

§14.5. Catamorphism in $\mathcal{S}$et

(14.5a)

name	definition	note
`listr`	`listr A ::= nil | cons (A, listr A)`	The cons-lists over `A`, the datatype every row below folds (B&dM p. 55).
`sum`	`⦇[zero, plus]⦈`	`plus(a, b) = a + b`.
`length`	`⦇[zero, outr succ]⦈`	`outr` drops the head and keeps the count of the tail, `succ` adds one for the head.
`average`	`⟨sum, length⟩ div`	`div(m, n) = m/n`, with `div(0, 0) = 0` so `average` is total. Traverses the list twice.
banana-split law	`⟨⦇h⦈, ⦇k⦈⟩ = ⦇⟨F(outl) h, F(outr) k⟩⦈`	Any fork of folds is a single fold, hence one traversal — `F` the base functor.
what it reduces to	`α ⟨⦇h⦈, ⦇k⦈⟩ = F(⟨⦇h⦈, ⦇k⦈⟩) ⟨F(outl) h, F(outr) k⟩`	All that (14.1a) leaves to check: the fork satisfies the defining equation, and uniqueness finishes it.
the instance	`⟨sum, length⟩ = ⦇[zeros, pluss]⦈`	`zeros = ⟨zero, zero⟩` and `pluss(a, (b, n)) = (a + b, n + 1)`, so `average` runs in one pass.
`preds`	`preds : Nat -> [Nat]`, `preds n = [n, n−1, …, 1]`	B&dM Ex 3.6 (p. 57), posed and not answered there: apply Ex 3.4 to write `preds` as `⦇k⦈ outl`.



$$\langle (\!|h|\!),\ (\!|k|\!) \rangle = (\!|\langle F(outl)\ h,\ F(outr)\ k \rangle|\!)$$ (14.5b)

The other laws.

The two $\sqsubseteq$ **fusion** rows rewrite $(\!|\alpha_B|\!)\ S$ through the same α_C and S. They are the only rows here that need the allegory **locally complete** — every hom-set a complete lattice — because they come from a least-fixed-point argument; the rest, the equality fusion (14.4a) included, needs only the initial algebra and the defining equation.

(14.5c)

name	**law**	**what it says**
Lambek	$\alpha_T{}^\circ = \alpha_T{}^{-1}$, the initial algebra is an isomorphism	A constructor can be undone — $\alpha_T{}^\circ$ takes a value apart into the parts it was built from.
Eilenberg–Wright	$\frac{(\!\mid\alpha_B\mid\!)}{\ni} = (\!\mid\frac{F(\ni)\ \alpha_B}{\ni}\mid\!)$	A relational fold is a deterministic fold of SETS: $\frac{\ }{\ni}$ pushes the nondeterminism into the power-object, where the fold is a map again.
Eilenberg–Wright	$(\!\mid\alpha_B\mid\!) = (\!\mid\frac{F(\ni)\ \alpha_B}{\ni}\mid\!)\ \ni$	The same fact read back — fold deterministically into a set, then take a member of it.
fusion	$(\!\mid\alpha_C\mid\!) \sqsubseteq (\!\mid\alpha_B\mid\!)\ S \Longleftarrow F(S)\ \alpha_C \sqsubseteq \alpha_B\ S$	Half of fusion: an inclusion between the two algebras is inherited by the folds.
fusion	$(\!\mid\alpha_B\mid\!)\ S \sqsubseteq (\!\mid\alpha_C\mid\!) \Longleftarrow \alpha_B\ S \sqsubseteq F(S)\ \alpha_C$	The other half, with both inclusions turned around.

The two $\frac{\ }{\ni}$ rows, drawn:

$$\begin{tikzcd}
F\ T \arrow[r, dashed, "F(K)"] \arrow[d, "\alpha_T"'] & F(E\ B) \arrow[r, "F(\ni)"] \arrow[d, "\frac{F(\ni)\ \alpha_B}{\ni}"] & F\ B \arrow[d, "\alpha_B"] \\
T \arrow[r, dashed, "K"'] & E\ B \arrow[r, "\ni"'] & B
\end{tikzcd}$$ (14.5d)

The left square is that catamorphism's own defining square, $K = (\!|\frac{F(\ni)\ \alpha_B}{\ni}|\!)$, and the right one is $\frac{\ }{\ni}$'s cancellation, so the outer rectangle says $K\ \ni$ satisfies the defining equation of $(\!|\alpha_B|\!)$ — and uniqueness finishes it.

§14.5.1. Fokkinga's mutual recursion theorem

- `F-Alg(𝒜)` — the algebras and their homomorphisms — has binary products whenever 𝒜 does, and the forgetful `U : F-Alg(𝒜) → 𝒜` **creates** them: the product of `h : F A → A` and `k : F B → B` is carried by `A × B`, with structure `⟨F(outl) h, F(outr) k⟩ : F(A × B) → A × B`.
- `outl` and `outr` are homomorphisms out of it, and `⟨p, q⟩ : X → A × B` is a homomorphism **iff** `p` and `q` both are — substitute the structure and compare the two forks component by component.
- The banana-split law of (14.5a) IS that product's universal property read at the initial algebra: `⦇h⦈` and `⦇k⦈` are the unique homomorphisms to `(A, h)` and `(B, k)`, so their fork is the unique homomorphism into the product, which is `⦇⟨F(outl) h, F(outr) k⟩⦈`.
- Fokkinga's theorem is **strictly more general** and is not that product: in (14.5.1a) the two arrows leave `F(A × B)`, not `F A` and `F B`, so each sees BOTH components. The product is the case where they factor as `F(outl) h` and `F(outr) k`; B&dM's Ex 3.4 is the other special case (p. 58). What it says: **any algebra on a product carrier is folded by a fork.**

$$
\begin{tikzcd}
\mathtt{F\ T} \arrow[r, dashed, "\mathtt{F(\langle f,\ g\rangle)}"] \arrow[d, "\alpha"'] & \mathtt{F(A \times B)} \arrow[d, "\mathtt{h}"] \\
\mathtt{T} \arrow[r, dashed, "\mathtt{f}"'] & \mathtt{A}
\end{tikzcd}
\qquad
\begin{tikzcd}
\mathtt{F\ T} \arrow[r, dashed, "\mathtt{F(\langle f,\ g\rangle)}"] \arrow[d, "\alpha"'] & \mathtt{F(A \times B)} \arrow[d, "\mathtt{k}"] \\
\mathtt{T} \arrow[r, dashed, "\mathtt{g}"'] & \mathtt{B}
\end{tikzcd}
$$

$$\mathtt{\alpha\ f = F(\langle f,\ g\rangle)\ h} \ \wedge\ \mathtt{\alpha\ g = F(\langle f,\ g\rangle)\ k} \ \equiv\ \mathtt{\langle f,\ g\rangle} = ⦇\mathtt{\langle h,\ k\rangle}⦈ \tag{14.5.1a}$$

§14.5.2. Ruby triangles

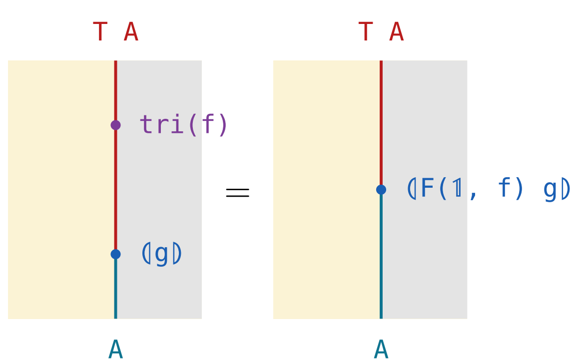
(14.5.2a)

stage	definition
informally (p. 58)	$\mathtt{tri(f)}\ [a_0, a_1, \ldots, a_i, \ldots, a_n] = [a_0, f\ a_1, \ldots, f^i\ a_i, \ldots, f^n\ a_n]$
for cons-lists (p. 59)	`tri(f) = ⦇[nil, (𝟙 × listr(f)) cons]⦈`
with the base functor named	`F(A, B) = 1 + A × B`, `α = [nil, cons]`, so `tri(f) = ⦇F(𝟙, listr(f)) α⦈`
abstractly (p. 59)	`F` a bifunctor with initial type `(α, T)`: `tri(f) = ⦇F(𝟙, T(f)) α⦈`

For the definition to make sense `f : A → A` is required, and then `tri(f) : T A → T A`.

(14.5.2b)

$$
\begin{tikzcd}
\mathtt{T\ A} \arrow[rr, "\mathtt{tri(f)}"] \arrow[dr, dashed, "⦇\mathtt{F(𝟙,\ f)\ g}⦈"'] & & \mathtt{T\ A} \arrow[dl, dashed, "⦇\mathtt{g}⦈"] \\
& \mathtt{A} &
\end{tikzcd}
$$



$$\mathtt{tri(f)}\ ⦇\mathtt{g}⦈ = ⦇\mathtt{F(𝟙,\ f)\ g}⦈ \quad\Longleftarrow\quad \mathtt{g\ f = F(f,\ f)\ g}$$

(14.5.2b) is Horner's rule, generalised from cons-lists to any initial type; B&dM call it that because for cons-lists it is the schoolbook method for evaluating a polynomial (p. 58). Fusion reduces it to `F(𝟙, T(f)) α ⦇g⦈ = F(𝟙, ⦇g⦈) F(𝟙, f) g` (p. 59).

§14.5.3. Depth of a tree

(14.5.3a)

the datatype	`tree A ::= tip A | bin (tree A, tree A)`, base functor `F(A, B) = A + B × B`, initial type `([tip, bin], tree)`
the fold	`⦇[g, h]⦈` is the unique `f` with `f (tip a) = g a` and `f (bin (x, y)) = h (f x, f y)`
`tree(f)`	`tree(f) = ⦇F(f, 𝟙) [tip, bin]⦈`; pointwise `tree(f) (tip a) = tip (f a)` and `tree(f) (bin (x, y)) = bin (tree(f) x, tree(f) y)`
`max`	`max = ⦇[𝟙, bmax]⦈`, where `bmax (a, b)` is the larger of `a` and `b`
`depths`	`depths = tree(zero) tri(succ)` — replaces every tip by its depth in the tree, `zero` the constant function returning 0 and `succ` the successor
`depth`	`depth = depths max`

§15. Type functor

(15a)

F a **binary** relator: $F(R, S)$ is its action on a pair, and $F(X)$ abbreviates $F(\mathbb{1}, X)$, the F of the catamorphism section. For every object A the initial algebra is $\alpha : F(A, T\ A) \to T\ A$, among the maps. The **type functor** T acts on an arrow $R : A \to B$ by

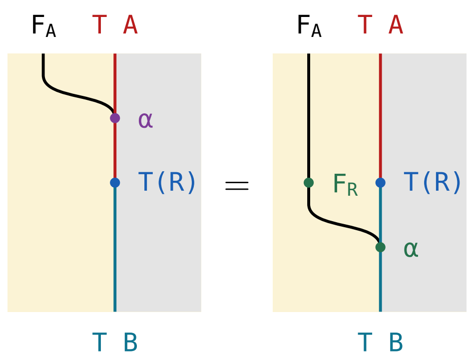
$$T(R) = \llparenthesis F(R, \mathbb{1})\ \alpha \rrparenthesis : T\ A \to T\ B$$

F is the **base functor** — one layer of the structure, acting on the recursive position — while T is the datatype itself, acting on the parameter. For cons-lists, $\mathtt{list}(R) = \llparenthesis[\mathtt{nil}, (R \otimes \mathbb{1})\ \mathtt{cons}]\rrparenthesis$, which is $\mathtt{map}(R)$.

A bifunctor puts its two arguments in two different places. The wire is the partial application $F_A = F(A, -)$, and an arrow in the SECOND argument is a bead on the object wire with the F wire running past it — the catamorphism section's rule, unchanged — while an arrow $R : A \to B$ in the FIRST induces a natural transformation $F_R : F_A \Rightarrow F_B$, which is a bead ON the F wire. Marsden draws bifunctors exactly so (`CatString.pdf`, **Bifunctors**): side-by-side is already spent on functor composition, so one argument has to go into the wire's name.

(15b)

$$\begin{tikzcd}
F(A, T\ A) \arrow[r, dashed, "F(\mathbb{1}, T(R))"] \arrow[d, "\alpha"'] & F(A, T\ B) \arrow[d, "F(R, \mathbb{1})\ \alpha"] \\
T\ A \arrow[r, dashed, "T(R)"'] & T\ B
\end{tikzcd}$$



$T(R) = \llparenthesis F(R, \mathbb{1})\ \alpha \rrparenthesis$

$T(R)$ is the unique arrow making it commute; there is no $\sqsubseteq$ in it.

Two beads at one height on two different wires are one action, here $F(R, T(R))$, so the right panel reads $F(R, T(R))\ \alpha$ and the left one $\alpha\ T(R)$. Their relative height says nothing: slide F_R below the $T(R)$ bead and the panel reads $F(\mathbb{1}, T(R))\ F(R, \mathbb{1})$ and then α, which is the square's top row followed by its right vertical; slide it above and the panel reads $F(R, \mathbb{1})\ F(\mathbb{1}, T(R))$, the route through $F(B, T\ A)$ the square leaves out. Identifying those two is the exchange condition every bifunctor satisfies (Hinze & Marsden, Ex 1.23), and in this calculus there is nothing to identify.

(15c)

name	**law**	**what it says**
the defining equation	$T(R) = \llparenthesis F(R, \mathbb{1})\ \alpha \rrparenthesis$	Rebuild the structure with α, applying R to the parameter on the way; that is $\mathtt{map}(R)$.
functor	$T(\mathbb{1}) = \mathbb{1}$ and $T(R)\ T(S) = T(R\ S)$	Mapping the identity changes nothing, and two maps in a row are one map.
type functor fusion	$T(R)\ \llparenthesis Q \rrparenthesis = \llparenthesis F(R, \mathbb{1})\ Q \rrparenthesis$	A map followed by a fold is a single fold — the mapped structure is never built.
naturality of α	$\alpha\ T(R) = F(R, T(R))\ \alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $G(R) = F(R, T(R))$ to T.
type relator	$T(R)^\circ = T(R^\circ)$	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

Type functor fusion is the equality fusion (14.4a) applied to $T(R)$'s own defining algebra, whose side condition holds because F is a bifunctor — $F(R, \mathbb{1})\ F(\mathbb{1}, \llparenthesis Q \rrparenthesis) = F(R, \llparenthesis Q \rrparenthesis) = F(\mathbb{1}, \llparenthesis Q \rrparenthesis)\ F(R, \mathbb{1})$ — so it too needs only the initial algebra and the defining equation.

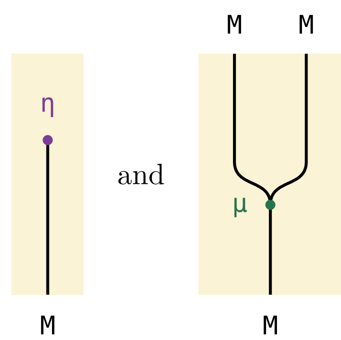
$$\begin{tikzcd}
\mathtt{F(A,\ T\ A)} \arrow[r, "\mathtt{F(R,\ T(R))}"] \arrow[d, "\alpha"'] & \mathtt{F(B,\ T\ B)} \arrow[d, "\alpha"] \\
\mathtt{T\ A} \arrow[r, "\mathtt{T(R)}"'] & \mathtt{T\ B}
\end{tikzcd}$$ (15d)

It commutes strictly, and it is (15b) with the right vertical's two halves separated: push `F(R, 𝟙)` up into the top row, where it meets `F(𝟙, T(R))` as the one action `F(R, T(R))`, and `α` is left as the vertical. In the string diagram that is the $\mathtt{F}_\mathtt{R}$ bead sliding up past `T(R)`, and what stays put is `α`, one bead falling past `T(R)` from the row above to the row below — which is what "`α` is natural" means.

§16. Monad

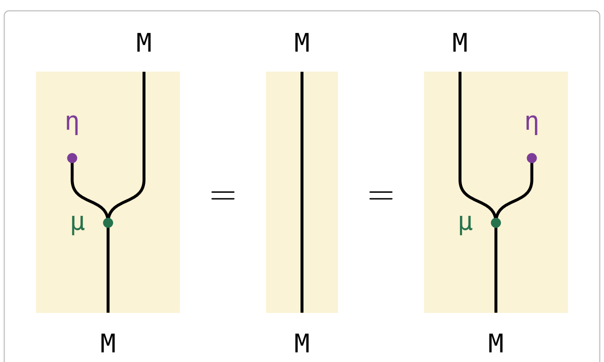
(16a)

> An endofunctor `M : 𝓐 → 𝓐` with two natural transformations, the **unit** `η : Id ⇒ M` and the **multiplication** `μ : M∘M ⇒ M`, subject to a unit law and an associativity law:
>
> `(η∘M) μ = 𝟙 = (M∘η) μ`   `(μ∘M) μ = (M∘μ) μ`



(16b)

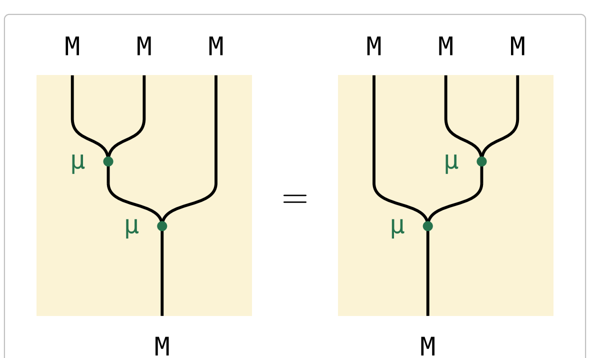
In (16b) the identity functor is a REGION and not a wire, so `η : Id ⇒ M` has nothing above it and the `M` wire simply begins at its bead — the book's lollipop. `μ : M∘M ⇒ M` is the two `M` wires of `M∘M` merging into one, its tuning fork.



(16c)

`(η∘M) μ = 𝟙 = (M∘η) μ`

The lollipop is absorbed into the fork from either side and a bare wire is left, which is what `𝟙` is in this calculus: nothing is drawn for it.



(16d)

`(μ∘M) μ = (M∘μ) μ`

The two nestings of the fork agree, so three `M`s collapse to one whichever pair is multiplied first.

§16.1. Monad in 𝒮et

monad	η	μ	η on elements	μ on elements
$\mathtt{Id\ A = A}$	$\mathbb{1}$	$\mathbb{1}$	$\mathtt{\eta\ A\ a = a}$	$\mathtt{\mu\ A\ a = a}$
$\mathtt{Exc\ A = A + E}$	ι_l	$\mathbb{1} \triangledown \iota_r$	$\mathtt{\eta\ A\ a} = \iota_l\ \mathtt{a}$	$\mathtt{\mu\ A}\ (\iota_l\ \mathtt{y}) = \mathtt{y}$ $\mathtt{\mu\ A}\ (\iota_r\ \mathtt{e}) = \iota_r\ \mathtt{e}$
$\mathtt{State\ A = (A \times P)^P}$	$\mathtt{curry}\ \mathbb{1}$	$(\mathtt{apply}_{\mathtt{A \times P}})^{\mathtt{P}}$	$\mathtt{\eta\ A\ a = \lambda\ x\ .\ (a,\ x)}$	$\mathtt{\mu\ A\ f = \lambda\ x\ .\ g\ y}$ where $\mathtt{(g,\ y) = f\ x}$
$\mathtt{Pow\ A = E\ A}$	$\frac{\mathbb{1}}{\ni}$	$\bigcup$	$\mathtt{\eta\ A\ a = \{a\}}$	$\mathtt{\mu\ A\ Xs} = \bigcup\ \mathtt{Xs}$

(16.1a)

§16.2. The state monad

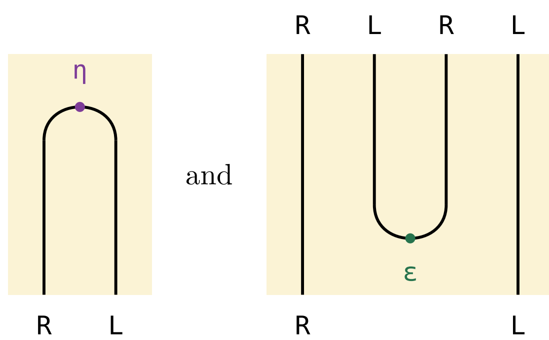
For a fixed set $\mathtt{P}$ of states, IntroString p. 67 defines

$$\mathtt{State\ A = (A \times P)^P} \qquad \eta\ \mathtt{A} = \mathtt{curry}\ (\mathbb{1} : \mathtt{A \times P \to A \times P}) \qquad \mu\ \mathtt{A} = (\mathtt{apply}_{\mathtt{A \times P}})^{\mathtt{P}}$$

Both arrows come from the curry adjunction $\mathtt{L} \dashv \mathtt{R}$, $\mathtt{L\ A = A \times P}$ and $\mathtt{R\ B = B^P}$ (IntroString p. 96):

- $\mathtt{State} = \mathtt{R}\circ\mathtt{L}$.
- η **is** the unit $\mathtt{Id} \Rightarrow \mathtt{R}\circ\mathtt{L}$, which is why it is the curried identity.
- the counit $\varepsilon : \mathtt{L}\circ\mathtt{R} \Rightarrow \mathtt{Id}$ is $\mathtt{apply}$.
- $\mu = \mathtt{R}\circ\varepsilon\circ\mathtt{L}$, that counit with a functor on each side: in $\mu\ \mathtt{A} = (\mathtt{apply}_{\mathtt{A \times P}})^{\mathtt{P}}$ the subscript $\mathtt{A \times P}$ is $\mathtt{L\ A}$, the functor INSIDE, and the outer $(-)^{\mathtt{P}}$ is $\mathtt{R}$, the functor OUTSIDE.
- $\dashv$ composes nothing; it only names the left adjoint first.

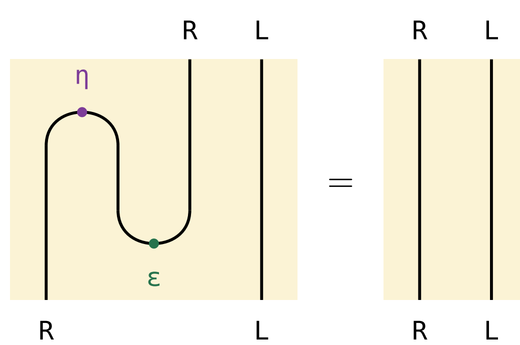
Applicative order stands $\mathtt{R}$ left and $\mathtt{L}$ right in $\mathtt{State} = \mathtt{R}\circ\mathtt{L}$. The identity functor is a region, so a unit $\mathtt{Id} \Rightarrow \mathtt{R}\circ\mathtt{L}$ has nothing above it and two wires below — one strand that OPENS the pair; a counit $\mathtt{L}\circ\mathtt{R} \Rightarrow \mathtt{Id}$ is that strand upside down, CLOSING a pair that arrives from above.



(16.2a)

In (16.2a), $\mathtt{State}\circ\mathtt{State}$ is the four wires $\mathtt{R\ L\ R\ L}$, and $\mu = \mathtt{R}\circ\varepsilon\circ\mathtt{L}$ closes the middle pair — the $\mathtt{L}\circ\mathtt{R}$ — leaving the outer $\mathtt{R}$ and $\mathtt{L}$, which is $\mathtt{State}$ again. The two wires that run past the turn untouched are the two functors composed on, $\mathtt{R}$ outside and $\mathtt{L}$ inside.

$$\begin{tikzcd} \mathtt{State\ A} \arrow[r, "\eta\ (\mathtt{State\ A})"] \arrow[dr, "\mathbb{1}"'] & \mathtt{State\ (State\ A)} \arrow[d, "\mu\ \mathtt{A}"] \\ & \mathtt{State\ A} \end{tikzcd}$$



$(\eta\circ\mathtt{State})\ \mu = \mathbb{1}$

(16.2b)

(16.2b) is the zig-zag. $\eta\circ\mathtt{State}$ opens a new $\mathtt{R\ L}$ left of the pair already there and μ closes the middle two — the new $\mathtt{L}$ against the old $\mathtt{R}$ — leaving the new $\mathtt{R}$ and the old $\mathtt{L}$, which is $\mathtt{State}$ unchanged. Strip the old $\mathtt{L}$, which never moves, and what zig-zags is the curry adjunction's snake equation $(\eta\circ\mathtt{R})(\mathtt{R}\circ\varepsilon) = \mathbb{1}$ (IntroString p. 93): (16c) read at $\mathtt{State}$ is that triangle identity with $\mathtt{L}$ composed on the inside.

In Set it is beta-reduction. $\eta\ \mathtt{(State\ A)\ f = \lambda\ x\ .\ (f,\ x)}$ pairs the computation with the state, and $\mu\ \mathtt{A}$ applies it back, $\mathtt{\lambda\ x\ .\ f\ x}$, which is $\mathtt{f}$.

§17. Combinatorial functions

(17a)

name	definition	note
`list`	$\texttt{list A ::= nil} \mid \texttt{cons (A, list A)}$	B&dM's `listr` under the short name it keeps from p. 125 on.
`subseq`	`⦇[nil, cons ∪ outr]⦈`	`xs subseq ys`: `ys` is `xs` with elements dropped — `cons` keeps the head, `outr` drops it.
`prefix`	`⦇[nil, nil ∪ cons]⦈ = cat° outl = init*`	`ys` is an initial segment of `xs`; the first `nil` is where it stops early. `init ≜ snoc° outl`.
`suffix`	`cat° outr = tail*`	The dual, `tail ≜ cons° outr`; as a catamorphism it needs snoc-lists.
`segment`	`suffix prefix`	A contiguous stretch of `xs`: a suffix, then a prefix of that.
`partition`	`concat°`, `concat ≜ ⦇[nil, cat]⦈`	`cat` restricted to a non-empty first argument, so `ys` is a list of non-empty segments of `xs`.
`inits`, `tails`	implementations of $\frac{\texttt{prefix}}{\ni}$ and $\frac{\texttt{suffix}}{\ni}$	`inits` by increasing length, `tails` by decreasing — opposite orders, which is what `splits ≜ ⟨inits, tails⟩ zip` needs to implement $\frac{\texttt{cat°}}{\ni}$.
`filter(p)`	$\frac{\texttt{subseq list(p)}}{\ni}$ `max R`, `R ≜ length ≤ length°`	The longest subsequence of `xs` whose every element passes `p`. `p` is a **coreflexive** and `list(p)` the relator applied to it, so `list(p)` is the test "every element passes" — no boolean anywhere. The longest is unique: only one subsequence keeps exactly the passing elements. The program is `⦇[nil, (outl p → cons, outr)]⦈`. Ex 7.41; `max R` is (18.1a)
`takewhile(p)`	$\frac{\texttt{prefix list(p)}}{\ni}$ `max R`	The same with `prefix` for `subseq`: the longest prefix whose every element passes `p`. Unique, since the prefixes of one list are linearly ordered by length. Ex 7.39
`mss`	$\frac{\texttt{segment sum}}{\ni}$ `max ≤`	Maximum segment sum. `segment = suffix prefix` splits it into $\frac{\texttt{prefix sum}}{\ni}$ `max ≤` on each suffix, which the greedy theorem turns into a catamorphism. Ex 7.40

§18. Optimisation Problems

§18.1. Minimum and maximum

For R : A → A, min R ≜ ∋ ∩ (∈\R°) : E A → A, max R ≜ min R°. (18.1a)

xs (min R) x ⟺ x ∈ xs ∧ (∀y ∈ xs. x R y)

xs (min R) x ⟺ (x in xs) and all x R\: xs in q

min R = ∋ ∩ all R° all R ≜ ∈\R, q's all; the chains below keep it written ∈\

the law	what it says
X ⊑ min R ⟺ X ⊑ ∋ and X° ∋ ⊑ R	in the set, and below every element of it
$\frac{1}{\ni}$ (∈\R) = R	bounding a singleton is bounding its element
$\frac{S}{\ni}$ (∈\R) = S°\R	bound S's image without building the set
union (∈\R) = ∈\(∈\R) union ≜ $\frac{\ni\ \ni}{\ni}$: E(E A) → E A	bound a union by bounding each member set
$\frac{1}{\ni}$ min R = 1 ∩ R	a singleton's minimum is its element, where R is reflexive $\frac{S}{\ni}$ min R at S := 1
$\frac{S}{\ni}$ min R = S ∩ (S°\R°)	an S-value that points to every S-value
$\frac{S}{\ni}$ min R = $\frac{S}{\ni}$ min(R ∩ S° S)	only R between values S gives one argument counts — context
E(S) min R = (∋ S) ∩ ((∋ S)°\R°)	the same for the image of a set $\frac{S}{\ni}$ min R at S := ∋ S
P(f) min R = min(f R f°) f	shunt a function through a minimum
P(S) min R = (∋ S) ∩ (∈\(S R°)) R reflexive	fusion with the power relator ⊒ is the only proof here that tabulates
P(S) min R ⊑ (∋ S) ∩ (∈\(S R°))	the half of the row above that costs nothing
P(min R) min R ⊑ union min R R a preorder	a minimum in each set, then a minimum of those
P(min R) min R = P(Dom (min R)) union min R R a preorder	the same as an equality, once empty sets are dropped

(18.1b)

§18.1.1. X ⊑ min R ⟺ X ⊑ ∋ and X° ∋ ⊑ R

$$\boxed{\frac{X}{\min R}} \underset{\Delta\dashv\cap}{\Longleftrightarrow} \begin{array}{c}\boxed{\frac{X}{\ni}}\\ \boxed{\frac{X}{\in\backslash R^\circ}}\end{array} \underset{\top\cdot\dashv\top\backslash}{\Longleftrightarrow} \begin{array}{c}\boxed{\frac{X}{\ni}}\\ \boxed{\frac{\in\ X}{R^\circ}}\end{array} \underset{\circ}{\Leftrightarrow} \begin{array}{c}\boxed{\frac{X}{\ni}}\\ \boxed{\frac{X^\circ\ \ni}{R}}\end{array} \tag{18.1.1a}$$

§18.1.2. $\frac{1}{\ni}$ (∈\R) = R

$$\boxed{\frac{X}{\frac{1}{\ni}\ (\in\backslash R)}} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \boxed{\frac{\frac{1}{\ni}{}^\circ\ X}{\in\backslash R}} \underset{\top\cdot\dashv\top\backslash}{\Longleftrightarrow} \boxed{\frac{\in\ \frac{1}{\ni}{}^\circ\ X}{R}} \underset{\circ}{\Leftrightarrow} \boxed{\frac{(\frac{1}{\ni}\ \ni)^\circ\ X}{R}} \underset{i\dashv E}{\Longleftrightarrow} \boxed{\frac{X}{R}} \tag{18.1.2a}$$

§18.1.3. $\frac{S}{\ni}$ (∈\R) = S°\R

$$\boxed{\frac{X}{\frac{S}{\ni}\ (\in\backslash R)}} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \boxed{\frac{\frac{S}{\ni}{}^\circ\ X}{\in\backslash R}} \underset{\top\cdot\dashv\top\backslash}{\Longleftrightarrow} \boxed{\frac{\in\ \frac{S}{\ni}{}^\circ\ X}{R}} \underset{\circ}{\Leftrightarrow} \boxed{\frac{(\frac{S}{\ni}\ \ni)^\circ\ X}{R}} \underset{\cdot\ni\dashv\frac{}{\ni}}{\Longleftrightarrow} \boxed{\frac{S^\circ\ X}{R}} \underset{\top\cdot\dashv\top\backslash}{\Longleftrightarrow} \boxed{\frac{X}{S^\circ\backslash R}} \tag{18.1.3a}$$

§18.1.4. union (∈\R) = ∈\(∈\R)

$$\boxed{\frac{X}{\mathrm{union}\ (\in\backslash R)}} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \boxed{\frac{\mathrm{union}^\circ\ X}{\in\backslash R}} \underset{\top\cdot\dashv\top\backslash}{\Longleftrightarrow} \boxed{\frac{\in\ \mathrm{union}^\circ\ X}{R}} \underset{\circ}{\Leftrightarrow} \boxed{\frac{(\mathrm{union}\ \ni)^\circ\ X}{R}} \underset{\cdot\ni\dashv\frac{}{\ni}}{\Longleftrightarrow} \boxed{\frac{(\ni\ \ni)^\circ\ X}{R}} \underset{\circ,\ \top\cdot\dashv\top\backslash}{\Longleftrightarrow} \boxed{\frac{X}{\in\backslash(\in\backslash R)}} \tag{18.1.4a}$$

§18.1.5. $\frac{S}{\ni}$ min R = S ∩ (S°\R°)

(18.1.5a)

$$\frac{X}{\frac{S}{\ni}\ \mathtt{min}\ R} \xLeftrightarrow[f^\circ\cdot\dashv f\cdot]{} \frac{\frac{S}{\ni}{}^\circ\ X}{\mathtt{min}\ R} \xLeftrightarrow[\Delta\dashv\cap]{} \begin{array}{c}\frac{\frac{S}{\ni}{}^\circ\ X}{\ni} \\ \frac{\frac{S}{\ni}{}^\circ\ X}{\in\backslash R^\circ}\end{array} \xLeftrightarrow[f^\circ\cdot\dashv f\cdot]{} \begin{array}{c}\frac{X}{\frac{S}{\ni}\ \ni} \\ \frac{X}{\frac{S}{\ni}\ (\in\backslash R^\circ)}\end{array} \xLeftrightarrow[\cdot\ni\dashv_{\frac{}{\ni}},\ (18.1.3a)]{} \begin{array}{c}\frac{X}{S} \\ \frac{X}{S^\circ\backslash R^\circ}\end{array} \xLeftrightarrow[\Delta\dashv\cap]{} \frac{X}{S\cap(S^\circ\backslash R^\circ)}$$

§18.1.6. $\frac{S}{\ni}$ min R = $\frac{S}{\ni}$ min(R ∩ S° S)

(18.1.6a)

$$\frac{X}{\frac{S}{\ni}\ \mathtt{min}(R\cap S^\circ\ S)} \xLeftrightarrow[(18.1.5a),\ ^\circ]{} \frac{X}{S\cap(S^\circ\backslash(R^\circ\cap S^\circ\ S))} \xLeftrightarrow[\Delta\dashv\cap,\ \top\cdot\dashv\top\backslash]{} \begin{array}{c}\frac{X}{S} \\ \frac{S^\circ\ X}{R^\circ\cap S^\circ\ S}\end{array}$$

$$\xLeftrightarrow[\Delta\dashv\cap,\ S^\circ\cdot\ \text{monotone}]{} \begin{array}{c}\frac{X}{S} \\ \frac{S^\circ\ X}{R^\circ}\end{array} \xLeftrightarrow[\top\cdot\dashv\top\backslash,\ \Delta\dashv\cap]{} \frac{X}{S\cap(S^\circ\backslash R^\circ)} \xLeftrightarrow[(18.1.5a)]{} \frac{X}{\frac{S}{\ni}\ \mathtt{min}\ R}$$

§18.1.7. P(f) min R = min(f R f°) f

(18.1.7a)

$$\underset{\text{f a map}}{P(f)\ \mathtt{min}\ R} = \!\!=_{P\ =\ E\ \text{on maps, (18.1.5a)}} (\ni\ f)\cap((\ni\ f)^\circ\backslash R^\circ) = \!\!=_{^\circ,\ \top\cdot\dashv\top\backslash,\ f^\circ\cdot\dashv f\cdot} (\ni\ f)\cap(\in\backslash(f\ R^\circ))$$

$$= \!\!=_{\text{modular law, f simple}} (\ni\cap((\in\backslash(f\ R^\circ))\ f^\circ))\ f = \!\!=_{\cdot f\dashv\cdot f^\circ,\ ^\circ,\ \mathtt{min}} \mathtt{min}(f\ R\ f^\circ)\ f$$

§18.1.8. P(S) min R = (∋ S) ∩ (∈\(S R°))

(18.1.8a)

> (p, q) tabulates W ≜ (∋ S) ∩ (∈\(S R°)), and y ≜ $\frac{(p\ \ni\ S)\ \cap\ (q\ R)}{\ni}$, a map.

(18.1.8b)

$$\frac{W}{P(S)\ \mathtt{min}\ R} \xLeftarrow[p^\circ\ q\ =\ W,\ \mathbb{1}\ \sqsubseteq\ y\ y^\circ]{} \begin{array}{c}\frac{p^\circ\ y}{P(S)} \\ \frac{y^\circ\ q}{\mathtt{min}\ R}\end{array}$$

(18.1.8c)

$$\frac{y^\circ\ q}{\ni} \xLeftarrow[f^\circ\cdot\dashv f\cdot,\ \cdot\ni\dashv_{\frac{}{\ni}}]{} \frac{q}{(p\ \ni\ S)\cap(q\ R)} \xLeftarrow[\Delta\dashv\cap,\ f^\circ\cdot\dashv f\cdot]{} \begin{array}{c}\frac{p^\circ\ q}{\ni\ S} \\ \underset{\text{R reflexive}}{\frac{\mathbb{1}}{R}}\end{array}$$

$$\frac{\in\ y^\circ\ q}{R^\circ} \xLeftarrow[\cdot\ni\dashv_{\frac{}{\ni}},\ ^\circ,\ \text{meet}]{} \underset{\text{q simple}}{\frac{R^\circ\ q^\circ\ q}{R^\circ}}$$

(18.1.8d)

$$\frac{p^\circ\ y}{P(S)} \xLeftarrow[\Delta\dashv\cap,\ \cdot\top\dashv/\top,\ ^\circ,\ f^\circ\cdot\dashv f\cdot]{} \begin{array}{c}\frac{p^\circ\ y\ \ni}{\ni\ S} \\ \frac{p\ \ni}{y\ \ni\ S^\circ}\end{array}$$

$$\frac{p^\circ\ y\ \ni}{\ni\ S} \xLeftarrow[\cdot\ni\dashv_{\frac{}{\ni}},\ \text{meet}]{} \underset{\text{p simple}}{\frac{p^\circ\ p\ \ni\ S}{\ni\ S}}$$

$$\frac{p\ \ni}{y\ \ni\ S^\circ} \xLeftarrow[\text{modular law}]{} \frac{p\ \ni}{(p\ \ni)\cap(q\ R\ S^\circ)} \xLeftarrow[\mathbb{1}\ \sqsubseteq\ q\ q^\circ]{} \frac{q^\circ\ p\ \ni}{R\ S^\circ} \xLeftarrow[^\circ,\ \top\cdot\dashv\top\backslash]{} \underset{\text{W's right half}}{\frac{W}{\in\backslash(S\ R^\circ)}}$$

§18.1.9. P(S) min R ⊑ (∋ S) ∩ (∈\(S R°))

(18.1.9a)

$$\frac{\texttt{P(S) min R}}{\texttt{(∋ S) ∩ (∈\textbackslash(S R°))}} \xLeftarrow[\Delta\dashv\cap,\ T\cdot\dashv T\backslash]{} \frac{\texttt{P(S) min R}}{\texttt{∋ S}}\ \ \frac{\texttt{∈ P(S) min R}}{\texttt{S R°}} \xLeftarrow[\text{UP of min}]{} \frac{\texttt{P(S) ∋}}{\texttt{∋ S}}\ \ \frac{\texttt{∈ P(S)}}{\texttt{S ∈}}$$

§18.1.10. P(min R) min R ⊑ union min R

(18.1.10a)

$$\underset{\text{R a preorder}}{\frac{\texttt{P(min R) min R}}{\texttt{union min R}}} \xLeftrightarrow[\text{(18.1.5a)}]{} \frac{\texttt{P(min R) min R}}{\texttt{(∋ ∋) ∩ ((∋ ∋)° \textbackslash R°)}} \xLeftarrow[°,\ \Delta\dashv\cap,\ T\cdot\dashv T\backslash]{} \frac{\texttt{P(min R) min R}}{\texttt{∋ ∋}}\ \ \frac{\texttt{∈ ∈ P(min R) min R}}{\texttt{R°}} \xLeftarrow[\text{UP of min, R transitive}]{} \frac{\texttt{P(min R) ∋}}{\texttt{∋ min R}}\ \ \frac{\texttt{∈ P(min R)}}{\texttt{min R ∈}}$$

§18.1.11. P(min R) min R = P(Dom (min R)) union min R

(18.1.11a)

$$\boxed{\texttt{P(min R) min R}} \xlongequal[\texttt{Dom (min R) min R = min R}]{} \boxed{\texttt{P(Dom (min R) min R) min R}}$$

$$\xlongequal[\text{P a relator}]{} \boxed{\texttt{P(Dom (min R)) P(min R) min R}} \ \underset{\text{(18.1.10a)}}{\sqsubseteq}\ \boxed{\texttt{P(Dom (min R)) union min R}}$$

(18.1.11b)

$$\frac{\texttt{P(Dom (min R)) union min R}}{\texttt{P(min R) min R}} \xLeftarrow[\text{(18.1.8b) at S := min R},\ \Delta\dashv\cap,\ T\cdot\dashv T\backslash]{} \frac{\texttt{P(Dom (min R)) union min R}}{\texttt{∋ min R}}\ \ \frac{\texttt{∈ P(Dom (min R)) union min R}}{\texttt{min R R°}}$$

$$\frac{\texttt{P(Dom (min R)) union min R}}{\texttt{∋ min R}} \xLeftarrow[\texttt{P(Dom (min R))} \sqsubseteq \mathbb{1}]{} \frac{\texttt{union min R}}{\texttt{∋ min R}}$$

$$\xLeftarrow[T(U \cap V) \sqsubseteq T\ U \cap T\ V,\ \cdot\ni\dashv_{\ni},\ (18.1.4a)]{} \frac{\texttt{(∋ ∋) ∩ (∈\textbackslash(∈\textbackslash R°))}}{\texttt{∋ min R}} \xLeftarrow[\text{modular law}]{} \underset{\text{counit of } T\cdot\dashv T\backslash}{\frac{\texttt{∋ (∋ ∩ (∈ (∈\textbackslash(∈\textbackslash R°))))}}{\texttt{∋ (∋ ∩ (∈\textbackslash R°))}}}$$

$$\frac{\texttt{∈ P(Dom (min R)) union min R}}{\texttt{min R R°}} \xLeftarrow[\in\ \text{lax natural}]{} \frac{\texttt{Dom (min R) ∈ union min R}}{\texttt{min R R°}}$$

$$\xLeftarrow[\texttt{Dom (min R)} \sqsubseteq \texttt{min R (min R)°}]{} \frac{\texttt{min R (min R)° ∈ union min R}}{\texttt{min R R°}} \xLeftarrow[\texttt{(min R)°} \sqsubseteq \in,\ \in\ \in\ \texttt{union} \sqsubseteq \in]{} \underset{\text{UP of min}}{\frac{\texttt{min R ∈ min R}}{\texttt{min R R°}}}$$

§18.2. Lax natural transformations

$\varphi_A : G\ A \to F\ A$ is a **lax natural transformation** $\varphi : G \Rightarrow F$ when, for every $R : A \to B$, $G(R)\ \varphi \sqsubseteq \varphi\ F(R)$. (5.13) (18.2a)

$\ni\ :\ P \Rightarrow Id$ — that is $P(R)\ \ni\ \sqsubseteq\ \ni\ R$, (13f)'s first row at $X := P(R)$.

It is enough to check **maps**, and there it is an equality $G(f)\ \varphi = \varphi\ F(f)$: laxness is about relations only. Theorem 5.2

$$\begin{tikzcd} G\ A \arrow[r, "\varphi"] \arrow[d, "G(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & F\ A \arrow[d, "F(R)"] \\ G\ B \arrow[r, "\varphi"'] & F\ B \end{tikzcd}$$ (18.2b)

$G(R)\ \varphi \sqsubseteq \varphi\ F(R)$

§18.3. Monotonic algebras

An F-algebra $S : F\ A \to A$ is **monotonic on** $R : A \to A$ if $F(R)\ S \sqsubseteq S\ R$. (18.3a)

For a map $f : F\ A \to A$ that is $f^\circ\ F(R)\ f \sqsubseteq R$, equivalently $F(R) \sqsubseteq f\ R\ f^\circ$.

$(\leq \times \leq)\ +\ \sqsubseteq\ +\ \leq$ — addition on Nat is monotonic on $\leq$, which at the point level reads $c = a + b \land a \leq a' \land b \leq b' \implies c \leq a' + b'$.

$F(R)\ S \sqsubseteq S\ R$ is (18.2a)'s inequation at $G := F$, $F := Id$, $\varphi := S$, so (18.3b) is (18.2b) with $Id(R)$ written R.

$$\begin{tikzcd} F\ A \arrow[r, "S"] \arrow[d, "F(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & A \arrow[d, "R"] \\ F\ A \arrow[r, "S"'] & A \end{tikzcd}$$ (18.3b)

$F(R)\ S \sqsubseteq S\ R$

$f : F\ A \to A$ **distributes over** R if $F(min\ R)\ f \sqsubseteq \frac{F(\ni)\ f}{\ni}\ min\ R$. (18.3c)

$+$ distributes over $\leq$, at the point level $min\ xs + min\ ys = min\{x + y \mid x \in xs \land y \in ys\}$ for xs, ys non-empty and $min \triangleq min\ \leq$.

$$\begin{tikzcd} F(E\ A) \arrow[r, "\frac{F(\ni)\ f}{\ni}"] \arrow[d, "F(min\ R)"'] \arrow[dr, phantom, "\sqsubseteq"] & E\ A \arrow[d, "min\ R"] \\ F\ A \arrow[r, "f"'] & A \end{tikzcd}$$ (18.3d)

$F(min\ R)\ f \sqsubseteq \frac{F(\ni)\ f}{\ni}\ min\ R$

$$\begin{tikzcd} \underset{(xs,\ ys)}{E\ Nat \times E\ Nat} \arrow[r, "\frac{(\ni\ \times\ \ni)\ +}{\ni}"] \arrow[d, "min\ \leq\ \times\ min\ \leq"'] \arrow[dr, phantom, "\sqsubseteq"] & \underset{\{x + y \mid x \in xs \land y \in ys\}}{E\ Nat} \arrow[d, "min\ \leq"] \\ \underset{(min\ xs,\ min\ ys)}{Nat \times Nat} \arrow[r, "+"'] & \underset{min\ xs + min\ ys}{Nat} \end{tikzcd}$$

$(min\ \leq\ \times\ min\ \leq)\ +\ \sqsubseteq \frac{(\ni\ \times\ \ni)\ +}{\ni}\ min\ \leq$

the law	what it says
$f^\circ\ F(R)\ f \sqsubseteq R \iff f^\circ\ F(R^\circ)\ f \sqsubseteq R^\circ$	a **map** is monotonic on an order and on its opposite together — a relation is not
$f^\circ\ F(R)\ f \sqsubseteq R \iff F(min\ R)\ f \sqsubseteq \frac{F(\ni)\ f}{\ni}\ min\ R$ Theorem 7.1, f a map	for a map the two definitions above are one condition
$(\!\lvert \frac{S}{\ni}\ min\ R \rvert\!) \sqsubseteq \frac{(\!\lvert S \rvert\!)}{\ni}\ min\ R$ Theorem 7.2, R a preorder and $F(R)\ S \sqsubseteq S\ R$	the **greedy theorem**: keeping one minimum at every step beats no more than taking the minimum at the end

(18.3e)

§18.4. Planning a company party

(18.4a)

$\mathtt{tree\ A ::= node\ (A,\ list(tree\ A))}$, base functor $\mathtt{F(A,\ B) = A \times list\ B}$, $\mathtt{rating : Employee \to Real}$.

$\mathtt{cost} \triangleq \mathtt{list(rating)\ sum},\ \mathtt{R} \triangleq \mathtt{cost} \le \mathtt{cost}^\circ,\ \mathtt{choose} \triangleq \mathtt{outl} \cup \mathtt{outr}$.

$\mathtt{include} \triangleq (\mathbb{1} \times (\mathtt{list(outr)\ concat}))\ \mathtt{cons} \quad \mathtt{exclude} \triangleq (\mathbb{1} \times (\mathtt{list(choose)\ concat}))\ \mathtt{outr}$

$\mathtt{S} \triangleq \langle \mathtt{include},\ \mathtt{exclude} \rangle,\ \mathtt{party} \triangleq \llparenthesis \mathtt{S} \rrparenthesis\ \mathtt{choose} : \mathtt{tree\ Employee} \to \mathtt{list\ Employee}$

(18.4b)

the law	what it says
$\frac{\mathtt{party}}{\ni}\ \mathtt{max\ R}$	the specification: a guest list of greatest total conviviality
$(\mathtt{R}^\circ \times \mathtt{R}^\circ)\ \mathtt{choose} \sqsubseteq \mathtt{choose\ R}^\circ$	$\mathtt{choose}$ is monotonic on $\mathtt{R}^\circ$ — the first claim
$(\mathbb{1} \times \mathtt{list}(\mathtt{R}^\circ \times \mathtt{R}^\circ))\ \mathtt{S} \sqsubseteq \mathtt{S}\ (\mathtt{R}^\circ \times \mathtt{R}^\circ)$	$\mathtt{S}$ is monotonic on $\mathtt{R}^\circ \times \mathtt{R}^\circ$ — the second claim
$\frac{\mathtt{party}}{\ni}\ \mathtt{max\ R} \sqsupseteq \llparenthesis \frac{\mathtt{S}}{\ni}\ \mathtt{max(R \times R)} \rrparenthesis\ \frac{\mathtt{choose}}{\ni}\ \mathtt{max\ R}$	the greedy theorem: one best pair per subtree instead of all parties of the whole tree
$\mathtt{party} = \llparenthesis \mathtt{S} \rrparenthesis\ \frac{\mathtt{choose}}{\ni}\ \mathtt{max\ R}$ $\mathtt{include} = (\mathbb{1} \times (\mathtt{list(outr)\ concat}))\ \mathtt{cons}$ $\mathtt{exclude} = \mathtt{outr\ list}(\frac{\mathtt{choose}}{\ni}\ \mathtt{max\ R})\ \mathtt{concat}$	the program, $\mathtt{exclude}$ renamed: $\mathtt{include}$ was already a map, and $\mathtt{exclude}$ becomes one

§18.5. Shortest paths on a cylinder

(18.5a)

$\mathtt{F(A,\ X) = A + A \times X}$, $\mathtt{L} = \mathtt{list}^+$ with initial algebra $\alpha : \mathtt{F(A,\ L\ A)} \to \mathtt{L\ A}$, $\mathtt{N}$ the $\mathtt{n}$-tuple relator.

$\mathtt{R} \triangleq \mathtt{sum} \le \mathtt{sum}^\circ,\ \mathtt{setify} : \mathtt{N\ A} \to \mathtt{E\ A},\ \mathtt{moves} : \mathtt{N\ A} \to \mathtt{E(N\ A)},\ \mathtt{trans} : \mathtt{E(N\ A)} \to \mathtt{N(E\ A)},\ \mathtt{zip} : \mathtt{F(N\ A,\ N\ B)} \to \mathtt{N\ F(A,\ B)},\ \mathtt{cp} \triangleq \frac{\mathtt{F}(\mathbb{1},\ \ni)}{\ni}$.

$\mathtt{generate} \triangleq \mathtt{F}(\mathbb{1},\ \mathtt{moves\ trans\ N(union)})\ \mathtt{zip\ N(cp\ P}(\alpha)) : \mathtt{F(N\ A,\ N(E(L\ A)))} \to \mathtt{N(E(L\ A))}$

$\mathtt{paths} \triangleq \llparenthesis \mathtt{generate} \rrparenthesis\ \mathtt{setify\ union} : \mathtt{L\ N\ Nat} \to \mathtt{E(L\ Nat)}$

(18.5b)

the law	what it says
$\mathtt{paths\ min\ R}$	the specification: a cheapest path across the cylinder
$\mathtt{F}(\mathbb{1},\ \mathtt{min\ R})\ \alpha \sqsubseteq \mathtt{cp\ P}(\alpha)\ \mathtt{min\ R}$ (7.13), Theorem 7.1 at the map α, using $\frac{\mathtt{F}(\mathbb{1},\ \ni)\ \alpha}{\ni} = \mathtt{cp\ P}(\alpha)$	extending every path in a set and then taking a minimum is beaten by extending one minimum
$\mathtt{generate\ N(min\ R)} \sqsupseteq \mathtt{F}(\mathbb{1},\ \mathtt{N(min\ R)})\ \mathtt{Q}$	the fusion condition that defines $\mathtt{Q}$ — this is what (7.13) is spent on
$\mathtt{Q} = \mathtt{F}(\mathbb{1},\ \mathtt{moves\ trans\ N(min\ R)})\ \mathtt{zip\ N}(\alpha)$ $= [\mathtt{N(wrap)},$ $(\mathbb{1} \times \mathtt{moves\ trans\ N(min\ R)})\ \mathtt{zip'\ N(cons)}]$	$\mathtt{generate}$ with the minimum taken inside each tuple component
$\mathtt{paths\ min\ R} \sqsupseteq \llparenthesis \mathtt{Q} \rrparenthesis\ \mathtt{setify\ min\ R}$	the program: one fold, $\mathtt{n}$ best paths carried per column

§18.6. The security van problem

(18.6a)

$\mathtt{R} \triangleq \mathtt{length} \le \mathtt{length}^\circ,\ \mathtt{ceiling} \triangleq \frac{\mathtt{prefix\ sum}}{\ni}\ \mathtt{max} \le,\ \mathtt{floor} \triangleq \frac{\mathtt{prefix\ sum}}{\ni}\ \mathtt{min} \le$.

$\mathtt{secure}$ the coreflexive on $\mathtt{x}$ with $\mathtt{bmax(ceiling\ x,\ ceiling\ x - floor\ x)} \le \mathtt{N}$; $\mathtt{ok}$ the coreflexive on $\mathtt{(a,\ xs)}$ with $\mathtt{xs}$ non-empty and $\mathtt{[a]} \mathbin{+\!\!+} \mathtt{head\ xs}$ secure.

$\mathtt{new} \triangleq (\mathtt{wrap} \times \mathbb{1})\ \mathtt{cons} \quad \mathtt{glue} \triangleq (\mathbb{1} \times \mathtt{cons}^\circ)\ \mathtt{assocl}\ (\mathtt{cons} \times \mathbb{1})\ \mathtt{cons}$

$\mathtt{old} \triangleq (\mathbb{1} \times \mathtt{cons}^\circ)\ \mathtt{assocl}\ ((\mathtt{cons\ secure}) \times \mathbb{1})\ \mathtt{cons}$

$\mathtt{partition} = \llparenthesis [\mathtt{nil},\ \mathtt{new} \cup \mathtt{glue}] \rrparenthesis,\ \mathtt{S} \triangleq [\mathtt{nil},\ \mathtt{new} \cup \mathtt{old}],\ \mathtt{partition\ list(secure)} = \llparenthesis \mathtt{S} \rrparenthesis$.

$\mathtt{H} \triangleq (\mathtt{head\ prefix}^\circ\ \mathtt{head}^\circ) \cup (\mathtt{nil}^\circ\ \mathtt{nil}),\ \mathtt{R} \mathbin{;} \mathtt{H} \triangleq \mathtt{R} \cap (\mathtt{R}^\circ \Rightarrow \mathtt{H}),\ |\mathtt{R}| \triangleq \mathtt{R} \cap \neg \mathtt{R}^\circ$.

(18.6b)

the law	**what it says**
$\frac{\texttt{partition list(secure)}}{\ni}$ `min R`	the specification: fewest secure segments
`secure prefix ⊑ prefix secure`	`secure` is prefix-closed — the only property of it used until the program
`(𝟙 × R) new ⊑ (new ∪ old) R` (7.14)	starting a new segment is monotonic on `R`
`(𝟙 × R) old ⊑ (new ∪ old) R` (7.15), FALSE	extending the first segment is not — the shorter partition need not stay secure
`(𝟙 × (R ; H)) new ⊑ (new ∪ old) (R ; H)` (7.16) `(𝟙 × (R ; H)) old ⊑ (new ∪ old) (R ; H)` (7.17)	both halves do become monotonic once `R` is refined by `H`, which breaks a tie in length by the first segment's prefix order
`(𝟙 × ⊤) new ⊑ new H` (7.18)	whatever the tail, `new`'s first segment is `[a]`, a prefix of every first segment — this is what (7.16) rests on
`(𝟙 × ⊤) old ⊑ new H` (7.19) `(𝟙 × |R|) old ⊑ new R` (7.20) `(𝟙 × (R ∩ H)) old ⊑ old (R ∩ H)` (7.21)	the three claims (7.17) rests on, split along `(R ; H) = |R| ∪ (R ∩ H)`
`old ⊑ new (R ; H)°`	`old` returns a shorter result than `new` if it returns any result at all, so the fold's choice refines to a map
`⦇[nil, (ok → glue, new)]⦈ ⊑ ⦇`$\frac{\texttt{S}}{\ni}$`min(R ; H)⦈`	the program: glue onto the first segment while it stays secure, else start a new one

§19. Thinning Algorithms

§19.1. Thinning

For $\mathtt{Q : A \to A}$, $\mathtt{thin\ Q} \triangleq \mathtt{(\ni/\ni) \cap (\in\backslash(Q^\circ\ \in)) : E\ A \to E\ A}$ (8.1). (19.1a)

$\mathtt{ys\ (thin\ Q)\ xs} \iff \mathtt{xs \subseteq ys \wedge (\forall a \in ys.\ \exists b \in xs.\ b\ Q\ a)}$

(19.1b)

the law	what it says
$\mathtt{X} \sqsubseteq \frac{\mathtt{S}}{\ni}\ \mathtt{thin\ Q} \iff \mathtt{X} \ni \sqsubseteq \mathtt{S}$ and $\mathtt{S^\circ\ X} \sqsubseteq \mathtt{Q^\circ} \in$	the universal property: everything kept is an S-value, and every S-value has a Q-lower bound among the kept ones
$\mathtt{Q} \sqsubseteq \mathtt{R} \Longrightarrow \mathtt{thin\ Q} \sqsubseteq \mathtt{thin\ R}$	the fewer pairs Q relates, the fewer subsets count as thinnings
$\mathbb{1} \sqsubseteq \mathtt{thin\ Q}$, and $\mathtt{thin\ Q}$ is a preorder if Q is	keeping everything is always a legal thinning
$\mathtt{min\ R = thin\ Q\ min\ R}$ $\mathtt{Q} \sqsubseteq \mathtt{R}$, both preorders	**thin-introduction**: thinning first cannot lose an R-minimum
$\mathtt{thin\ Q} \sqsupseteq \mathtt{min\ Q}\ \frac{\mathbb{1}}{\ni}$ (8.2)	**thin-elimination**: keeping one element is a thinning, but its domain is the sets min Q is defined on
$\frac{\mathtt{S}}{\ni}\ \mathtt{thin\ Q} \sqsupseteq \frac{\mathtt{S}}{\ni}\ \mathtt{min\ R}\ \frac{\mathbb{1}}{\ni}$ (8.3), $\mathtt{R} \cap \mathtt{(S^\circ\ S)} \sqsubseteq \mathtt{Q}$	the usable variant: R need only refine Q between values S gives one argument
$\mathtt{union\ thin\ Q} \sqsupseteq \mathtt{P(thin\ Q)\ union}$ (8.4)	thinning each member set is a thinning of the union
$\llparenthesis \frac{\mathtt{F(\ni)\ S}}{\ni}\ \mathtt{thin\ Q} \rrparenthesis \sqsubseteq \frac{\llparenthesis \mathtt{S} \rrparenthesis}{\ni}\ \mathtt{thin\ Q}$ Theorem 8.1, S monotonic on Q	the **thinning theorem**: thinning at every step beats thinning only at the end
$\llparenthesis \frac{\mathtt{F(\ni)\ S}}{\ni}\ \mathtt{thin\ Q} \rrparenthesis\ \mathtt{min\ R} \sqsubseteq \frac{\llparenthesis \mathtt{S} \rrparenthesis}{\ni}\ \mathtt{min\ R}$ Corollary 8.1, $\mathtt{Q} \sqsubseteq \mathtt{R}$ as well	the same against the optimisation problem itself, by thin-introduction

§19.2. Paths in a layered network

$\mathtt{F(A, X) = A + A \times X}$, $\mathtt{L = list^+}$ with initial algebra $\alpha \triangleq \mathtt{[wrap, cons] : F(A, L\ A) \to L\ A}$. (19.2a)

$\mathtt{wrapz} \triangleq \mathtt{\langle wrap, zero \rangle}$, $\mathtt{consw\ (a, (xs, n)) = (cons\ (a, xs), wt\ (a, head\ xs) + n)}$.

$\mathtt{cost} \triangleq \llparenthesis \mathtt{[wrapz, consw]} \rrparenthesis\ \mathtt{outr}$, $\llparenthesis \mathtt{[wrapz, consw]} \rrparenthesis = \langle \mathbb{1}, \mathtt{cost} \rangle$, $\mathtt{R} \triangleq \mathtt{cost \leq cost^\circ}$.

$\mathtt{Q} \triangleq \mathtt{R \cap (head\ head^\circ)}$, $\mathtt{S} \triangleq \mathtt{F(\mathbb{1}, \ni)}\ \alpha$, $\frac{\mathtt{F(\ni, \mathbb{1})}}{\ni} = \mathbb{1} + \mathtt{cpl}$, $\frac{\mathtt{F(\mathbb{1}, \ni)}}{\ni} = \mathbb{1} + \mathtt{cpr}$, $\mathtt{step} \triangleq \mathtt{cpr\ P(cons)\ min\ R}$.

(19.2b)

the law	what it says
$\frac{\llparenthesis \mathtt{F(\ni, \mathbb{1})}\ \alpha \rrparenthesis}{\ni}\ \mathtt{min\ R} = \frac{\mathtt{L(\ni)}}{\ni}\ \mathtt{min\ R}$	the specification: a least-cost path across the network
$\mathtt{F(\ni, Q)}\ \alpha \sqsubseteq \mathtt{F(\ni, \mathbb{1})}\ \alpha\ \mathtt{Q}$	S is monotonic on Q; on R it is not, since the next edge can cost arbitrarily much
$\llparenthesis \frac{\mathtt{F(\ni, \ni)}\ \alpha}{\ni}\ \mathtt{thin\ Q} \rrparenthesis\ \mathtt{min\ R} \sqsubseteq \frac{\llparenthesis \mathtt{F(\ni, \mathbb{1})}\ \alpha \rrparenthesis}{\ni}\ \mathtt{min\ R}$ Corollary 8.1	thinning applies: one partial path per starting vertex is enough
$\mathtt{S\ head} \sqsubseteq \mathtt{[\mathbb{1}, outr]}$ S head simple, so $\mathtt{R} \cap \mathtt{(S^\circ\ S)} \sqsubseteq \mathtt{Q}$	the side condition of (8.3): between two paths S builds from one argument, equal cost and equal head already means Q
$\frac{\mathtt{F(\ni, \ni)}\ \alpha}{\ni}\ \mathtt{thin\ Q}$ $\sqsupseteq \frac{\mathtt{F(\ni, \mathbb{1})}}{\ni}\ \mathtt{P(}\frac{\mathtt{F(\mathbb{1}, \ni)}}{\ni}\ \mathtt{P(\alpha)\ min\ R)}$ (8.4), (8.3), $\mathtt{P(}\frac{\mathbb{1}}{\ni}\mathtt{)\ union} = \mathbb{1}$	thin eliminated: split the algebra at $\mathtt{F(\ni, \mathbb{1})\ F(\mathbb{1}, \ni)}$, thin each part, one minimum per part
$\frac{\mathtt{F(\mathbb{1}, \ni)}}{\ni}\ \mathtt{P(\alpha)\ min\ R = [wrap, step]}$	that inner part read off the two branches of α
$\frac{\llparenthesis \mathtt{F(\ni, \mathbb{1})}\ \alpha \rrparenthesis}{\ni}\ \mathtt{min\ R} \sqsupseteq \llparenthesis \mathtt{[P(wrap), cpl\ P(step)]} \rrparenthesis\ \mathtt{min\ R}$	the program: one fold, a best path per vertex of the current layer

§19.3. Implementing thin

(19.3a)

$\mathtt{setify : list\ A \to E\ A,\ cup : E\ A \times E\ A \to E\ A,\ cp(F)} \triangleq \frac{\mathtt{F(\ni)}}{\ni}\mathtt{,\ listcp(F) : F(list\ A) \to list(F\ A),\ sort\ P} \triangleq$ $\mathtt{setify^\circ\ ordered\ P}$ for $\mathtt{P}$ a connected preorder.

$\mathtt{thinlist\ Q}$ is any $\mathtt{thinlist\ Q \sqsubseteq subseq}$ with $\mathtt{thinlist\ Q\ setify \sqsubseteq setify\ thin\ Q}$; one is $\mathtt{(\![[nil,\ bump\ Q]]\!)}$, $\mathtt{bump\ Q\ (a,\ []) = [a],\ bump\ Q\ (a,\ [b] +\!\!+\ xs) = (b\ Q\ a \to [a] +\!\!+\ xs,\ a\ Q\ b \to [b] +\!\!+\ xs,\ [a] +\!\!+\ [b] +\!\!+\ xs)}$.

Binary thinning data: $\mathtt{S = (f_1\ p_1) \cup (f_2\ p_2)}$ with $\mathtt{p_1, p_2}$ coreflexive; $\mathtt{Q}$ a preorder with $\mathtt{Q \sqsubseteq R}$ and both $\mathtt{f_1\ p_1}$, $\mathtt{f_2\ p_2}$ monotonic on $\mathtt{Q}$; $\mathtt{P}$ a connected preorder with both $\mathtt{f_1, f_2}$ monotonic on $\mathtt{P}$; $\mathtt{g_i} \triangleq \mathtt{list(f_i)\ filter(p_i)}$.

(19.3b)

the law	what it says
$\mathtt{thinlist\ Q\ xs = [minlist\ Q\ xs]}$ (8.5), $\mathtt{Q}$ connected, $\mathtt{xs}$ non-empty	what thinning should come to when it can: one element
$\mathtt{sort\ P\ thinlist\ Q \sqsubseteq thin\ Q\ sort\ P}$ (8.6)	thinning a sorted list is a thinning of the set — this is what $\mathtt{thinlist\ Q \sqsubseteq subseq}$ buys
$\mathtt{sort\ P\ minlist\ Q \sqsubseteq min\ Q}$ (8.7)	a minimum of the sorted list is a minimum of the set
$\mathtt{sort(f\ P\ f^\circ)\ list(f) \sqsubseteq P(f)\ sort\ P}$ (8.8)	shunt a function through a sort
$\mathtt{sort\ P\ filter(p) \sqsubseteq E(p)\ sort\ P}$ (8.9), $\mathtt{p}$ coreflexive	filtering a sorted list sorts the restricted set
$\mathtt{(sort\ P \times sort\ P)\ merge\ P \sqsubseteq cup\ sort\ P}$ (8.10)	merging two sorted lists sorts their union
$\mathtt{F(sort\ P)\ listcp(F) \sqsubseteq cp(F)\ sort(F\ P)}$ (8.11), $\mathtt{F}$ linear	$\mathtt{listcp(F)}$ is the list implementation of the cartesian product $\mathtt{cp(F)}$
$\mathtt{F(sort\ P)\ listcp(F)\ list(f)\ filter(p)} \sqsubseteq \frac{\mathtt{F(\ni)\ f\ p}}{\ni}\ \mathtt{sort\ P}$ Lemma 8.1, $\mathtt{f}$ monotonic on $\mathtt{P}$, $\mathtt{p}$ coreflexive	one sorted list built from sorted arguments, instead of a set built and then sorted
$\mathtt{(\![listcp(F)\ \langle g_1,\ g_2\rangle\ merge\ P\ thinlist\ Q]\!)\ minlist\ R} \sqsubseteq \frac{(\![S]\!)}{\ni}\ \mathtt{min\ R}$ Theorem 8.2	the **binary thinning theorem**: a fold on sorted lists of partial solutions, thinned at every step

§19.4. The knapsack problem

(19.4a)

$\mathtt{vol,\ wt : Item \to Real,\ value} \triangleq \mathtt{list(vol)\ sum,\ weight} \triangleq \mathtt{list(wt)\ sum}$.

$\mathtt{subseq = (\![[nil,\ cons] \cup [nil,\ outr]]\!),\ within\ w}$ the coreflexive on $\mathtt{xs}$ with $\mathtt{weight\ xs \le w}$.

$\mathtt{\ge} \triangleq \mathtt{\le^\circ,\ R} \triangleq \mathtt{value \ge value^\circ,\ Q} \triangleq \mathtt{R \cap (weight \le weight^\circ),\ P} \triangleq \mathtt{R}$.

$\mathtt{F\ A = 1 + Item \times A,\ listcp(F) = wrap + cpr,\ g_1} \triangleq \mathtt{list([nil,\ cons])\ filter(within\ w) = [list(nil),\ h_1]}$, $\mathtt{g_2} \triangleq \mathtt{list([nil,\ outr]) = [list(nil),\ h_2]}$.

$\mathtt{h_1} \triangleq \mathtt{list(cons)\ filter(within\ w),\ h_2} \triangleq \mathtt{list(outr)}$.

(19.4b)

the law	what it says
$\frac{\mathtt{subseq\ (within\ w)}}{\ni}\ \mathtt{min\ R}$	the specification: a selection of greatest total value that stays within the capacity $\mathtt{w}$
$\mathtt{subseq\ (within\ w) = (\![[nil,\ cons]\ (within\ w)) \cup [nil,\ outr]]\!)}$ fusion, weights non-negative	the selections that fit are themselves a fold, and the fold's algebra already has binary thinning's shape
$\mathtt{(1\!\!1 \times R)\ (cons\ (within\ w)) \sqsubseteq cons\ (within\ w)\ R}$ FALSE	not monotonic on $\mathtt{R}$: a selection of greater value need not still fit once one more item goes in
$\mathtt{(1\!\!1 \times Q)\ (cons\ (within\ w)) \sqsubseteq cons\ (within\ w)\ Q}$ $\mathtt{(1\!\!1 \times Q)\ outr \sqsubseteq outr\ Q}$	both halves are monotonic on $\mathtt{Q}$ once ties in value are broken by weight
$\mathtt{(\![listcp(F)\ \langle g_1,\ g_2\rangle\ merge\ R\ thinlist\ Q]\!)\ minlist\ R}$ Theorem 8.2 at $\mathtt{P} \triangleq \mathtt{R}$, $\mathtt{F}$ linear	binary thinning, sorting in descending order of value
$\mathtt{knapsack\ w = (\![[nil,\ cpr\ \langle h_1,\ h_2\rangle\ merge\ R\ thinlist\ Q]]\!)\ minlist\ R}$	the program; $\mathtt{minlist\ R}$ becomes $\mathtt{head}$, since packings come out in descending value

§19.5. The paragraph problem

(19.5a)

$Line = list^{+}\ Word$, $Para = list^{+}\ Line$, $F\ A = Word + Word \times A$, $listcp(F) = wrap + cpr$.

$new\ (a, xs) = [[a]] \mathbin{+\!\!+} xs$, $glue\ (a, xs) = [[a] \mathbin{+\!\!+} head\ xs] \mathbin{+\!\!+} tail\ xs$, $partition \triangleq \llparenthesis [wrap\ wrap,\ new \cup glue] \rrparenthesis : list^{+}\ Word \to Para$.

$width \triangleq \llparenthesis [length,\ (length \times \mathbb{1})\ plus\ succ] \rrparenthesis$, $fits\ w$ the coreflexive on a line x with $width\ x \le w$, $ok\ w$ the coreflexive on $[x] \mathbin{+\!\!+} xs$ with $width\ x \le w$.

$white\ w\ x = w - width\ x$, $collect \triangleq list(sqr)\ sum$, $waste\ w \triangleq init\ list(white\ w)\ collect$.

$R \triangleq (waste\ w) \le (waste\ w)^{\circ}$, $Q \triangleq R \cap (head\ head^{\circ})$, $P \triangleq \top$.

$g_1 \triangleq list([wrap\ wrap,\ new])$, $g_2 \triangleq list([wrap\ wrap,\ glue])\ filter(ok\ w)$, $start \triangleq wrap\ wrap\ wrap$, $h_1 \triangleq list(new)$, $h_2 \triangleq list(glue)\ filter(ok\ w)$.

(19.5b)

the law	what it says
$\frac{partition\ list^{+}(fits\ w)}{\ni}\ min\ R$	the specification: least waste among the paragraphs whose every line fits
$partition\ list^{+}(fits\ w) = \llparenthesis [wrap\ wrap,\ new \cup (glue\ (ok\ w))] \rrparenthesis$ fusion, every word fits on a line by itself	the fitting paragraphs are themselves a fold
$[wrap\ wrap,\ new \cup (glue\ (ok\ w))]$ $= [wrap\ wrap,\ new] \cup ([wrap\ wrap,\ glue]\ (ok\ w))$	rewritten into binary thinning's shape $(f_1\ p_1) \cup (f_2\ p_2)$
$(\mathbb{1} \times R)\ glue \sqsubseteq glue\ R$ FALSE	$glue$ is not monotonic on R: the waste of a paragraph depends on its whole first line, so no greedy algorithm solves this
$(\mathbb{1} \times Q)\ new \sqsubseteq new\ Q$ $(\mathbb{1} \times Q)\ (glue\ (ok\ w)) \sqsubseteq glue\ (ok\ w)\ Q$ $cons$ monotonic on $collect \le collect^{\circ}$	both halves are monotonic on Q once ties in waste are broken by the first line
$merge\ \top = cat$; $P \triangleq head\ prefix\ head^{\circ}$ also serves	$\top$ needs no sorting at all, and $prefix$ is a linear order on first lines of paragraphs of one input
$\llparenthesis listcp(F)\ \langle g_1, g_2\rangle\ cat\ thinlist\ Q \rrparenthesis\ minlist\ R$ Theorem 8.2 at $P \triangleq \top$	binary thinning, one candidate kept per first line
$paragraph\ w = \llparenthesis [start,\ cpr\ \langle h_1, h_2\rangle\ cat\ thinlist\ Q] \rrparenthesis\ minlist\ R$	the program, g_1 and g_2 split along the coproduct

§19.6. Bitonic tours

(19.6a)

$F\ A = (City \times City) + (City \times A)$, the base functor of cons-lists of length at least two; $listcp(F) = wrap + cpr$; $tc : City \times City \to Real$, neither positive nor symmetric.

$start\ (a, b) = ([a, b], [a, b])$, $dropl\ (a, ([b] \mathbin{+\!\!+} xs, ys)) = ([a] \mathbin{+\!\!+} xs, [a] \mathbin{+\!\!+} ys)$, $dropr\ (a, (xs, [b] \mathbin{+\!\!+} ys)) = ([a] \mathbin{+\!\!+} xs, [a] \mathbin{+\!\!+} ys)$, $tour \triangleq \llparenthesis [start,\ dropl \cup dropr] \rrparenthesis$.

$cost\ (xs, ys) = outcost\ xs + incost\ ys$, $outcost\ [a_0, \ldots, a_n] = tc\ (a_0, a_1) + \cdots + tc\ (a_{n-1}, a_n)$, $incost\ [a_0, \ldots, a_n] = tc\ (a_1, a_0) + \cdots + tc\ (a_n, a_{n-1})$.

$next \triangleq tail\ head$, $next2 \triangleq next \times next$, $R \triangleq cost \le cost^{\circ}$, $Q \triangleq R \cap (next2\ next2^{\circ})$, $P \triangleq \top$, $g_1 \triangleq list([start,\ dropl])$, $g_2 \triangleq list([start,\ dropr])$.

(19.6b)

the law	what it says
$\frac{\mathtt{tour}}{\ni}$ `min R`	the specification: a least-cost bitonic tour, outward journey and return kept as a pair of lists
$(\mathbb{1} \times R)\ \mathtt{dropl} \sqsubseteq \mathtt{dropl}\ R$ FALSE $(\mathbb{1} \times R)\ \mathtt{dropr} \sqsubseteq \mathtt{dropr}\ R$ FALSE	neither drop is monotonic on `R`: the two edges it adds and removes depend on `head` and `next` of both lists
$(\mathbb{1} \times Q)\ \mathtt{dropl} \sqsubseteq \mathtt{dropl}\ Q$ $(\mathbb{1} \times Q)\ \mathtt{dropr} \sqsubseteq \mathtt{dropr}\ Q$	both are, once ties in cost are broken by the two second cities — the heads already agree among tours of one input
$(\!\lvert \mathtt{listcp}(F)\ \langle g_1, g_2 \rangle\ \mathtt{cat\ thinlist}\ Q \rvert\!)\ \mathtt{minlist}\ R$ Theorem 8.2 at $P \triangleq \top$, $\mathtt{merge}\ \top = \mathtt{cat}$	binary thinning, one candidate per pair of second cities
$\mathtt{mintour} = (\!\lvert [\mathtt{start\ wrap},\ \mathtt{cpr}\ \langle \mathtt{list(dropl)},$ $\mathtt{list(dropr)} \rangle\ \mathtt{cat\ thinlist}\ Q] \rvert\!)\ \mathtt{minlist}\ R$	the program: quadratic, because each step adds just two tours to the list kept

§20. Dynamic Programming

§20.1. Theory

$h : F\,B \to B$ a map, $T : F\,A \to A$ an F-algebra, $R : B \to B$. (20.1a)

$H \triangleq (\!|T|\!)^\circ\ (\!|h|\!) : A \to B$, $M \triangleq \frac{H}{\ni}\ \mathtt{min}\ R$ the problem to be solved, $(\mu X : G(X))$ the least fixed point of G.

(20.1b)

the law	what it says
$(\mu X : \frac{T^\circ}{\ni}\ P(F(X)\ h)\ \mathtt{min}\ R) \sqsubseteq M$ Theorem 9.1, h monotonic on R	**dynamic programming**: decompose in every way, solve each part, keep one optimum per part
$\frac{T^\circ}{\ni}\ P(F(M)\ h)\ \mathtt{min}\ R \sqsubseteq M$ (9.1)	all Knaster–Tarski leaves to prove
$\frac{T^\circ}{\ni}\ P(F(M)\ h)\ \mathtt{min}\ R \sqsubseteq H$ (9.2) $H^\circ\ \frac{T^\circ}{\ni}\ P(F(M)\ h)\ \mathtt{min}\ R \sqsubseteq R$ (9.3)	(9.1) split by the universal property of $\mathtt{min}$
$P(X)\ \mathtt{min}\ R \sqsubseteq (\ni\ X) \cap (\in\backslash(X\ R^\circ))$ (9.4) = (7.10), (18.1b)	the only fact about $\mathtt{min}$ either proof uses
$(\mu X : \frac{T^\circ}{\ni}\ \mathtt{thin}\ Q\ P(F(X)\ h)\ \mathtt{min}\ R) \sqsubseteq M$ Theorem 9.2, Q a preorder with $Q\ F(H)\ h \sqsubseteq F(H)\ h\ R$; $\mathtt{thin}\ Q$ as in (19.1b)	the same with a thinning step: decompositions that can never win are dropped
the fixed point is unique, and entire Theorem 6.3, T° followed by F's membership relation inductive; $\frac{T^\circ}{\ni}$ finite and non-empty, R connected	when the recursion can be refined to a recursive function
$\frac{[V_1,\ V_2]^\circ}{\ni}\ \mathtt{thin}(Q_1 + Q_2)\ P([U_1,\ U_2])\ \mathtt{min}\ R$ $= (\mathrm{Ran}\ V_1 \to W_1,\ W_2)$ $W_i \triangleq \frac{V_i{}^\circ}{\ni}\ \mathtt{thin}(Q_i)\ P(U_i)\ \mathtt{min}\ R$ Proposition 9.1, $V_2\ \ V_1{}^\circ = \bot$	$F\ A$ is usually a coproduct, and disjoint ranges split the fixed point into one branch per summand
$F(R)\ h \sqsubseteq h\ R$ $Q\ F(H)\ h \sqsubseteq F(H)\ h\ R$	the two conditions to be checked, in the order the three propositions below discharge them
$F(R)\ h \sqsubseteq h\ R$ Proposition 9.2, $R \triangleq \mathtt{cost} \le \mathtt{cost}^\circ$, $h\ \mathtt{cost} = F(\mathtt{cost})\ k$, $F(\le)\ k \sqsubseteq k \le$	monotonicity when the cost is itself a fold with a step k monotonic on $\le$
$F(R \cap (H^\circ\ H))\ h \sqsubseteq h\ R$ Proposition 9.3, $R \triangleq \mathtt{cost} \le \mathtt{cost}^\circ$, $h\ \mathtt{cost} = F(\langle \mathtt{cost},\ H^\circ \rangle)\ k$, $F(\le \times 𝟙)\ k \sqsubseteq k \le$, H° simple	monotonicity **in context**: k may also read the input the part was built from
$Q \triangleq F(U,\ V)$ Proposition 9.4, U, V preorders, $F(U,\ R)\ h \sqsubseteq h\ R$, $V\ H \sqsubseteq H\ R$	both conditions of Theorem 9.2 at once, split along the two arguments of a bifunctor

§20.2. The string edit problem

$\mathtt{Op} ::= \mathtt{cpy\ Char} \mid \mathtt{del\ Char} \mid \mathtt{ins\ Char}$, $F(A,\ B) = 1 + (A \times B)$, $\alpha \triangleq [\mathtt{nil},\ \mathtt{cons}]$. (20.2a)

$\mathtt{edit} \triangleq (\![\mathtt{base},\ \mathtt{step}]\!) : \mathtt{list\ Op} \to \mathtt{list\ Char} \times \mathtt{list\ Char}$, $\mathtt{base}$ returning $([],\ [])$, $\mathtt{step\ (cpy\ a,\ (xs,\ ys))} = ([a] + xs,\ [a] + ys)$, $\mathtt{step\ (del\ a,\ (xs,\ ys))} = ([a] + xs,\ ys)$, $\mathtt{step\ (ins\ a,\ (xs,\ ys))} = (xs,\ [a] + ys)$.

$\mathtt{length} \triangleq (\![\mathtt{zero},\ \mathtt{outr\ succ}]\!)$, $R \triangleq \mathtt{length} \le \mathtt{length}^\circ$, $U \triangleq \top$, $V \triangleq \mathtt{suffix}^\circ \times \mathtt{suffix}^\circ$, $Q \triangleq 𝟙 + (U \times V)$, $\mathtt{empty}$ the coreflexive on $(xs,\ ys)$ with both lists empty.

$\mathtt{unstep}$ implements $\frac{\mathtt{step}^\circ}{\ni}\ \mathtt{thin}(U \times V)$: $\mathtt{unstep\ ([a] + xs,\ [])} = [(\mathtt{del\ a},\ (xs,\ []))]$, $\mathtt{unstep\ ([],\ [b] + ys)} = [(\mathtt{ins\ b},\ ([],\ ys))]$, $\mathtt{unstep\ ([a] + xs,\ [b] + ys)} = (a = b \to [(\mathtt{cpy\ a},\ (xs,\ ys))],\ [(\mathtt{del\ a},\ (xs,\ [b] + ys)),\ (\mathtt{ins\ b},\ ([a] + xs,\ ys))])$.

(20.2b)

the law	what it says
$\frac{\mathtt{edit}^\circ}{\ni}$ `min R`	the specification: a shortest edit sequence from which both strings can be reconstituted
`F(R) α ⊑ α R` Proposition 9.2 at `length`, `succ` monotonic on ≤	`cons` is monotonic on `R`, so Theorem 9.1 already applies
`Q F(𝟙, edit°) α ⊑ F(𝟙, edit°) α R`	the thinning condition sought, over `F(Op, list Char × list Char)`
`F(⊤, R) α ⊑ α R` left as an exercise	`U ≜ ⊤` costs nothing: any two operations may be compared
`edit (𝟙 × suffix) ⊑ R° edit` `edit (suffix × 𝟙) ⊑ R° edit`	the other half of Proposition 9.4 for `V ≜ suffix° × suffix°`

`edit (𝟙 × tail) ⊑ R° edit`
`edit (tail × 𝟙) ⊑ R° edit`
`suffix = tail*`, and `B A ⊑ C B ⟹ B A* ⊑ C* B`
	one step is enough: drop the operation that produced the head, or weaken its `cpy` to a `del` — never lengthening the sequence

`(μX :` $\frac{\mathtt{[base,\ step]}^\circ}{\ni}$ `thin Q P([nil, (𝟙 × X) cons]) min R)`
`⊑` $\frac{\mathtt{edit}^\circ}{\ni}$ `min R`
Theorem 9.2
	a copy, when available, beats a delete or an insert

`X = (empty → nil,` $\frac{\mathtt{step}^\circ}{\ni}$ `thin(U × V) P((𝟙 × X) cons) min R)` Proposition 9.1	`base` and `step` have disjoint ranges
`mle = (empty → nil, unstep list((𝟙 × mle) cons) minlist R)`	the program: at most two decompositions survive, but the same subproblem is solved many times, so the running time is exponential
`column xs ys = [mle (u, ys) | u ← tails xs]` `column xs = ⦇[fstcol xs, nextcol xs]⦈, fstcol = list(del) tails`	the tabulation: `mle (xs, ys)` needs `mle (u, v)` for every tail `u` of `xs` and `v` of `ys`, so the columns are built right to left

`column xs ([b] ++ ys) = nextcol xs (b, column xs ys)`
`nextcol xs (b, us)`
`  = ⦇[base (b, last us), step b]⦈ xus`
`  xus = zip (xs, zip (init us, tail us))`
	each column is a fold built bottom to top, over `xs` zipped with the adjacent pairs of the column to its right

`base (b, u) = [[ins b] ++ u]`
`step b ((a, (u, v)), ws) =`
`  (a = b → [[cpy a] ++ v] ++ ws,`
`  [bmin R ([del a] ++ w, [ins b] ++ u)] ++ ws)`
`w = head ws`; these `base`, `step` are not `edit`'s
	an entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§20.3. Optimal bracketing

(20.3a)

`tree A ::= tip A | bin (tree A, tree A)`, `F X = A + X²`, so `F(R) = 𝟙 + R²`; `h ≜ [tip, bin]`, `flatten ≜ ⦇[wrap, cat]⦈ : tree A → list⁺ A`, `H = flatten°`.

`⟨cost, size⟩ ≜ ⦇[opt, opb]⦈`, `opt ≜ ⟨zero, st⟩`, `opb ((cx, sx), (cy, sy)) = (cb (sx, sy) + cx + cy, sb (sx, sy))`.

`sb` associative, so `size = flatten sz` for a map `sz`; `R ≜ cost ≤ cost°`, `g ≜ [zero, (𝟙 × sz)² opb outl]`, `single` the coreflexive on singleton lists.

`splits ≜ ⟨inits⁺, tails⁺⟩ zip`, an implementation of $\frac{\mathtt{cat}^\circ}{\ni}$; `array ≜ inits list(row)`, `row ≜ tails list(mct)`, `col ≜ inits list(mct)`.

`mix ≜ zip list(bin) minlist R`, `next ≜ ⟨outl, mix⟩ snoc`, `process ≜ ((tip wrap) × 𝟙) loop(next)`.

the law	what it says
$\frac{\mathtt{flatten}^\circ}{\ni}\ \mathtt{min}\ R$	the specification: a least-cost bracketing of $a_1 \oplus \cdots \oplus a_n$, the tree whose flattening is the given list
$[\mathtt{tip}, \mathtt{bin}]\ \mathtt{cost} = (\mathbb{1} + \langle \mathtt{cost}, \mathtt{flatten}\rangle^2)\ g$ (9.5)	the cost of a node reads only the cost and the flattening of its two subtrees
$(\mathbb{1} + (\le \times \mathbb{1})^2)\ g \sqsubseteq g\ \le$ (9.6)	g is monotonic on $\le$ in its two cost arguments
$F(R \cap (\mathtt{flatten}\ \mathtt{flatten}^\circ))\ h \sqsubseteq h\ R$ Proposition 9.3, $H^\circ = \mathtt{flatten}$ a map	monotonicity in context: only trees with the same flattening are compared
$(\mu X : \frac{[\mathtt{wrap}, \mathtt{cat}]^\circ}{\ni}\ P([\mathtt{tip}, (X \times X)\ \mathtt{bin}])\ \mathtt{min}\ R) \sqsubseteq \frac{\mathtt{flatten}^\circ}{\ni}\ \mathtt{min}\ R$ Theorem 9.1	split the list in every way, bracket both halves, join
$X = (\mathtt{single} \to \mathtt{wrap}^\circ\ \mathtt{tip}, \frac{\mathtt{cat}^\circ}{\ni}\ P((X \times X)\ \mathtt{bin})\ \mathtt{min}\ R)$ Proposition 9.1	`wrap` and `cat` have disjoint ranges
$\mathtt{mct} = (\mathtt{single} \to \mathtt{head}\ \mathtt{tip}, \mathtt{splits}\ \mathtt{list}((\mathtt{mct} \times \mathtt{mct})\ \mathtt{bin})\ \mathtt{minlist}\ R)$	the program; exponential, since the segments of one list overlap
$\mathtt{mct} = (\mathtt{single} \to \mathtt{head}\ \mathtt{tip}, \langle \mathtt{init}\ \mathtt{col}, \mathtt{tail}\ \mathtt{row}\rangle\ \mathtt{mix})$ (9.7)	the tabulation: `mct xs` is needed for every non-empty segment `xs`, so the values are held as an array of rows
$\mathtt{col} = (\mathtt{single} \to \mathtt{head}\ \mathtt{tip}\ \mathtt{wrap}, \langle \mathtt{init}\ \mathtt{col}, \mathtt{tail}\ \mathtt{row}\rangle\ \mathtt{next})$ (9.8) $\mathtt{cons}\ \mathtt{col} = (\mathbb{1} \times \mathtt{array})\ \mathtt{process}$ (9.9) $\mathtt{row} = (\mathtt{single} \to \mathtt{head}\ \mathtt{tip}\ \mathtt{wrap}, \langle \mathtt{mct}, \mathtt{tail}\ \mathtt{row}\rangle\ \mathtt{cons})$ (9.10)	a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop
$\mathtt{array} = (\![[\mathtt{fstcol}, \mathtt{addcol}]]\!)$, $\mathtt{fstcol} \triangleq \mathtt{tip}\ \mathtt{wrap}\ \mathtt{wrap}$ $\mathtt{addcol} \triangleq \langle \mathtt{outl}\ \mathtt{tip}\ \mathtt{wrap}, \mathtt{step}\rangle\ \mathtt{cons}$ $\mathtt{step} \triangleq \langle \mathtt{process}\ \mathtt{tail}, \mathtt{outr}\rangle\ \mathtt{zip}\ \mathtt{list}(\mathtt{cons})$	the program: one fold building the array column by column, cubic in the length of the input

(20.3b)

§20.4. Data compression

(20.4a)

$\mathtt{list}\ A ::= \mathtt{nil} \mid \mathtt{snoc}\ (\mathtt{list}\ A, A)$, $\mathtt{list}^+\ A ::= \mathtt{wrap}\ A \mid \mathtt{snoc}\ (\mathtt{list}^+\ A, A)$, $\mathtt{String} = \mathtt{list}\ \mathtt{Char}$, $\mathtt{Code} ::= \mathtt{sym}\ \mathtt{Char} \mid \mathtt{ptr}\ (\mathtt{String}, \mathtt{String}^+)$, $F(\mathtt{Code}, \mathtt{String}) = 1 + (\mathtt{String} \times \mathtt{Code})$, $\alpha \triangleq [\mathtt{nil}, \mathtt{snoc}]$.

$\mathtt{decode} \triangleq (\![[\mathtt{nil}, \mathtt{extend}]]\!) : \mathtt{list}\ \mathtt{Code} \to \mathtt{String}$, $\mathtt{extend}\ (xs, \mathtt{sym}\ a) = xs +\!\!+ [a]$, $\mathtt{extend}\ (xs, \mathtt{ptr}\ (ys, zs)) = xs +\!\!+ zs$ when $ys +\!\!+ zs$ is a proper prefix of $xs +\!\!+ zs$; $H = \mathtt{decode}^\circ$.

$\mathtt{size} \triangleq (\![[\mathtt{zero}, \mathtt{distr}\ [\mathbb{1} \times c, \mathbb{1} \times p]\ \mathtt{plus}]]\!)$ with c, p the constant costs of a symbol and a pointer; $R \triangleq \mathtt{size}\ \le\ \mathtt{size}^\circ$, $Q \triangleq F(\top + \top, \mathtt{prefix}^\circ) = \mathbb{1} + (\mathtt{prefix}^\circ \times (\top + \top))$, the two $\top$ on symbols and on pointers.

$\mathtt{lrt}\ ws = \mathtt{min}(\mathtt{prefix}^\circ \times (\top + \top))\ \{(xs, (ys, zs)) \mid xs +\!\!+ zs = ws, ys +\!\!+ zs$ a proper prefix of $ws\}$, the longest repeated tail; $\mathtt{reduce}\ (ws +\!\!+ [a]) = (zs \ne [\,] \to [(ws, \mathtt{sym}\ a), (xs, \mathtt{ptr}\ (ys, zs))], [(ws, \mathtt{sym}\ a)])$ with $(xs, (ys, zs)) = \mathtt{lrt}\ (ws +\!\!+ [a])$.

(20.4b)

the law	**what it says**
$\frac{\mathtt{decode}^\circ}{\ni}\ \mathtt{min}\ \mathtt{R}$	the specification: a smallest code sequence decoding to the given string
`F(R) α ⊑ α R` `(⊤ + ⊤) [c, p] = [c, p]`	monotonicity is routine, the two costs being constants

`F(⊤ + ⊤, R) α ⊑ α R`
`decode prefix ⊑ R° decode`
Proposition 9.4 at `Q ≜ F(⊤ + ⊤, prefix°)`
	the thinning condition split in two: pointers are ordered only by how much input they consume, symbols and pointers not at all

`decode init ⊑ R° decode`	enough for the second: drop the last character of the output by shortening or removing the last code element, never raising the cost
$(\mu X : \frac{\mathtt{[nil,\ extend]}^\circ}{\ni}\ \mathtt{thin}\ Q\ \mathtt{P([nil,\ (X \times 1\!\!1)\ snoc])\ min}\ R) \sqsubseteq \frac{\mathtt{decode}^\circ}{\ni}\ \mathtt{min}\ R$ Theorem 9.2	between a symbol and a pointer nothing can be decided in advance; between two pointers the longer match wins
$X = (\mathtt{null} \to \mathtt{nil},\ \frac{\mathtt{extend}^\circ}{\ni}\ \mathtt{thin(prefix^\circ \times (\top + \top))\ P((X \times 1\!\!1)\ snoc)\ min}\ R)$ Proposition 9.1	`nil` and `extend` have disjoint ranges
$\frac{\mathtt{extend}^\circ}{\ni}\ \mathtt{(ws ⧺ [a]) = \{(ws,\ sym\ a)\} \cup}$ `{(xs, ptr (ys, zs)) | xs ⧺ zs = ws ⧺ [a]`, `ys ⧺ zs` a proper prefix of `ws}`	the decompositions of one string: take the last character as a symbol, or end with a pointer
`reduce` implements $\frac{\mathtt{extend}^\circ}{\ni}\ \mathtt{thin(prefix^\circ \times (\top + \top))}$	thinning leaves at most two: the symbol, and the pointer of `lrt`
`encode = (null → nil, reduce list((encode × 𝟙) snoc) minlist R)`	the program, again exponential; the book gives no tabulation for it

§21. Greedy Algorithms

§21.1. Theory

> h, T, R, H, M as in (20.1a); additionally Q a **connected** preorder on the sets $\frac{T^\circ}{\ni}$ returns, so that $\frac{T^\circ}{\ni}$ min Q is entire. (21.1a)

(21.1b)

the law	what it says
(μX : $\frac{T^\circ}{\ni}$ min Q F(X) h) ⊑ M Theorem 10.1, the hypotheses of Theorem 9.2	**greedy**: one decomposition is kept at every step, so P and $\frac{}{\ni}$ disappear from the recursion
$\frac{[V_1,\ V_2]^\circ}{\ni}$ min(Q_1 + Q_2) [U_1, U_2] = (Ran V_1 → W_1, W_2) W_i ≜ $\frac{V_i{}^\circ}{\ni}$ min(Q_i) U_i Proposition 10.1, V_2 $V_1{}^\circ$ = ⊥	Proposition 9.1 with min for thin
Q ≜ F(U, V) Proposition 9.4	still the way to get both conditions, but $\frac{T^\circ}{\ni}$ min Q must now be entire as well
Theorem 7.2, (18.3e)	that greedy theorem chooses among the **results** of one relational step of a catamorphism; Theorem 10.1 chooses among the **decompositions** of the input, for an arbitrary T rather than an initial algebra, at the cost of h being a map

§21.2. The detab-entab problem

> detab ≜ ⦇[nil, expand]⦈ : String → String over snoc-lists, α ≜ [nil, snoc], H = detab°; expand (xs, a) = (a = TB → fill xs, xs ⧺ [a]), fill xs = xs ⧺ blanks (n − (col xs) mod n). (21.2a)
>
> col ≜ ⦇[zero, count]⦈, count (c, a) = (a = NL → 0, c + 1); TB the tab, BL the blank, NL the newline, tab stops every n columns.
>
> R ≜ length ≤ length°, U the preorder with a U b ⟺ a = TB ∨ a = b, V ≜ prefix° ∩ (fill fill°), Q ≜ 𝟙 + (V × U).
>
> unfill xs the shortest prefix of xs with fill (unfill xs) = fill xs; tbc the trailing blank count, triple ≜ ⟨unfill entab, ⟨tbc, col⟩⟩.

(21.2b)

the law	what it says
$\frac{\mathtt{detab}^\circ}{\ni}$ `min R`	the specification: a shortest input `detab` expands to the given output — `detab entab = 𝟙` and nothing shorter does
`F(T, R) α ⊑ α R` left as an exercise	`U` may be any preorder on characters; `a U b ⟺ a = TB ∨ a = b` puts `TB` below every character, so `min` prefers a tab to a blank
`detab prefix ⊑ R° detab` FALSE	at `n = 8`, `detab [a,b,c,d,e,TB] = [a,b,c,d,e,BL,BL,BL]`, whose prefix `[a,b,c,d,e,BL,BL]` is longer than any input giving it
`detab V° ⊑ R° detab` `V ≜ prefix° ∩ (fill fill°)`	true once only the prefixes that do not cross a tab stop are allowed
`nil V° = nil` `fill V° = fill` `snoc V° ⊑ snoc ∪ (outl V°)` left as exercises	the three properties of `V` the derivation rests on; the second is what `fill fill°` was added for
`expand V° ⊑ expand ∪ (outl V°)`	the claim they add up to: shortening the output either leaves the last step alone or discards it
`detab V° ⊑ detab ∪ (init detab V°)` `init` inductive, so the greatest solution is the unique one	hence `detab V° ⊑ prefix detab`, and `prefix ⊑ R°` finishes Proposition 9.4
`(μX :` $\frac{\mathtt{[nil,\ expand]}^\circ}{\ni}$ `min Q (𝟙 + (X × 𝟙)) [nil, snoc]) ⊑` $\frac{\mathtt{detab}^\circ}{\ni}$ `min R` Theorem 10.1	greedy: one character of input is decided at each step
`X = (null → nil,` $\frac{\mathtt{expand}^\circ}{\ni}$ `min(V × U) (X × 𝟙) snoc)` Proposition 10.1	`nil` and `expand` have disjoint ranges
$\frac{\mathtt{expand}^\circ}{\ni}$ `(xs ++ [a]) = {(ys, TB) | fill ys = xs ++ [a]} ∪ {(xs, a)}` the first set is non-empty iff `a = BL` and `col (xs ++ [a]) mod n = 0`	the last output character came from a tab only if it is a blank landing exactly on a tab stop
$\frac{\mathtt{expand}^\circ}{\ni}$ `min(V × U) (xs ++ [a]) =` `(a = BL ∧ col (xs ++ [a]) mod n = 0 → (unfill xs, TB), (xs, a))`	the greedy step: emit a tab whenever a tab is legal, consuming all the blanks back to the previous tab stop
`entab xs = entab (unfill xs) ++ blanks (tbc xs)` (10.1)	what makes `triple` a snoc-list catamorphism: the output splits at the last tab stop
`triple = ⦇[base, op]⦈` and `entab = triple assocl outl (𝟙 × blanks) cat`	the program: one pass carrying the column and the count of pending blanks

`base` returns `([], (0, 0))`
`op ((xs, (t, c)), a) =`
`(a = BL ∧ (c + 1) mod n ≠ 0 → (xs, (t + 1, c + 1)),`
`a = BL → (xs ++ [TB], (0, c + 1)),`
`a = NL → (xs ++ blanks t ++ [NL], (0, 0)),`
`(xs ++ blanks t ++ [a], (0, c + 1)))`
	hold a blank back, cash the held blanks in for a tab at a tab stop, flush them at a newline, flush them before anything else

§21.3. The minimum tardiness problem

(21.3a)

`F X = 1 + (X × Job)`, `α ≜ [nil, snoc]` on schedules, `β ≜ [nil, snag]` on bags, `snag` putting a job into a bag; `bagify ≜ ⦇β⦈ : list Job → Bag Job`, `H = bagify°`.

`ct, dt, wt : Job → Real` the completion, due and weighting quantities of a job; `penalty (xs, j) = (sum (list(ct) xs) + ct j − dt j) × wt j`.

`cost ≜` $\frac{\mathtt{prefix}}{\ni}$ `P(α° [zero, penalty]) max ≤`, `cost [] = 0`, `cost (xs ++ [j]) = bmax (cost xs, penalty (xs, j))`, `R ≜ cost ≤ cost°`.

`perm ≜ bagify bagify° = ⦇[nil, add]⦈`, `add (xs, j) = ys ++ [j] ++ zs` for some `xs = ys ++ zs`.

`k ≜ [zero, assocr (𝟙 × ((bagify° × 𝟙) penalty)) bmax]`, `f ≜ [zero, (bagify° × 𝟙) penalty]`, `Q ≜ f ≤ f°`.

(21.3b)

the law	what it says
$\frac{\texttt{bagify}^\circ}{\ni}$ `min R`	the specification: an ordering of the given bag of jobs with least maximum penalty
`F(R ∩ (bagify bagify°)) α ⊑ α R` (10.2)	monotonicity in context — only schedules of one bag are compared
`(Q ∩ (β β°)) F(bagify°) α ⊑ F(bagify°) α R` (10.3)	the greedy condition, also in context
`cost (xs ⧺ [j]) = bmax (cost xs, penalty (perm xs, j))` `α cost = F(⟨cost, bagify⟩) k, F(≤ × 𝟙) k ⊑ k ≤` Proposition 9.3	(10.2): `penalty` sums the completion times of `xs`, and a sum does not see the order
`α cost = ⟨g, h⟩ bmax` (10.4) `g ≜ [zero, penalty]` (10.5) `h ≜ [zero, outl cost]` (10.6)	the cost of a schedule split into the penalty of its last job and the cost of the rest
`add ⊑ outl R` (10.7) `β bagify° = F(bagify°) [nil, add]` (10.8)	adding a job never lowers the cost; and `bagify°` is itself a catamorphism on bags. Together: `β bagify° ⊑ F(bagify°) h ≤ cost°`
`F(bagify) Q° F(bagify°) = g ≥ g°`, met by `Q ≜ f ≤ f°, f ≜ [zero, (bagify° × 𝟙) penalty]`	the specification `Q` has to satisfy, and the choice that meets it: a `Q`-minimum is a job of least penalty
no greedy **catamorphism** exists	a greedy snoc-list catamorphism would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole
`X = (null → nil,` $\frac{\texttt{snag}^\circ}{\ni}$ `min Q (X × 𝟙) snoc)` Theorem 10.1, Proposition 10.1	`nil` and `snag` have disjoint ranges
`schedule = (null → nil, pick (schedule × 𝟙) snoc)` `pick ⊑` $\frac{\texttt{snag}^\circ}{\ni}$ `min Q`, a partial function	the program: repeatedly remove a job of least penalty and put it last; quadratic in the number of jobs

§21.4. The TeX problem

(21.4a)

`intern ≜ val round : Decimal → [0, 2¹⁶)`, `val ≜ ⦇[zero, shift]⦈`, `shift (d, r) = (d + r)/10`, `round r` rounds `2¹⁶ r` to the nearest integer: `round r = n ⟺ 2n − 1 < 2¹⁷ r < 2n + 1`.

`interval n = ((2n − 1)/2¹⁷, (2n + 1)/2¹⁷)`, `r inrange (a, b) ⟺ a < r < b`, `round° = interval inrange`, `R ≜ length ≤ length°`.

`Interval` the pairs `(a, b)` with `0 < b < 1` and `a < b` (10.9) `[arb, step] : 1 + (Digit × Interval) → Interval`, `step (d, (a, b)) = ((d + a)/10, (d + b)/10)`.

`F X = 1 + (Digit × X)`, `α ≜ [nil, cons]`, `h ≜ ⦇[arb, step]⦈ = H°`, `! : Digit × Interval → 1`, `Q ≜ (ιₗ° !° ιᵣ) ∪ 𝟙`, `w ≜ 2¹⁷`.

(21.4b)

the law	**what it says**
$\frac{\mathtt{intern}^\circ}{\ni}\ \mathtt{min\ R} = \mathtt{interval}\ \frac{\mathtt{inrange\ val}^\circ}{\ni}\ \mathtt{min\ R}$	the specification: a shortest decimal whose internal representation is the given multiple of 2^{-16}. $\mathtt{round}^\circ$ is not a map, but $\mathtt{interval}$ is, so it comes out of the $\frac{}{\ni}$
$\mathtt{val\ inrange}^\circ = (\!\lbrack \mathtt{[arb,\ step]} \rbrack\!)$ fusion; $\mathtt{zero\ inrange}^\circ = \mathtt{arb}$, $\mathtt{shift\ inrange}^\circ = (\mathbb{1} \times \mathtt{inrange}^\circ)\ \mathtt{step}$	the converse of $\mathtt{val}$, cut down to intervals, is a catamorphism on cons-lists
$F(R)\ \alpha \sqsubseteq \alpha\ R$	$\mathtt{cons}$ is monotonic on R — routine
$\alpha^\circ\ F(h)\ Q^\circ \sqsubseteq R^\circ\ \alpha^\circ\ F(h)$	the greedy condition of Theorem 10.1, converted
$\frac{\mathtt{arb}^\circ}{\ni}\ (a,\ b) = (a < 0 \to \{*\},\ \{\})$ $\frac{\mathtt{step}^\circ}{\ni}\ (a,\ b) = \{(d,\ (10a - d,\ 10b - d))\ \mid\ 0 < 10b - d < 1\}$	the decompositions of an interval: stop, or take one more digit
$0 < 10b - d_1 < 1$ and $0 < 10b - d_2 < 1$ imply $d_1 = d_2$	$\mathtt{step}^\circ$ is in fact a map, d the digit with $0 < 10b - d < 1$, so $\frac{\mathtt{[arb,\ step]}^\circ}{\ni}$ returns at most two elements and Q need only choose between them
$Q \triangleq (\iota_l^\circ\ !^\circ\ \iota_r) \cup \mathbb{1}$, and $!\ \mathtt{nil} \sqsubseteq \mathtt{cons}\ R^\circ$ from $!\ \mathtt{nil\ length} \sqsubseteq \mathtt{cons\ length}\ \geq$	stop whenever stopping is legal: the empty decimal is shorter than any other
$(\mu X : \frac{\mathtt{[arb,\ step]}^\circ}{\ni}\ \mathtt{min}\ Q\ F(X)\ \alpha) \sqsubseteq \frac{\mathtt{intern}^\circ}{\ni}\ \mathtt{min}\ R$ Theorem 10.1	greedy: emit the one digit the interval allows, until the interval contains zero
`extern = interval f` `f (a, b) = (a < 0 → [], [d] ⧺ f (10a − d, 10b − d))`	the program, with d the digit above

`extern n = f (2n − 1, 2n + 1)`
`f (p, q) = (p ≤ 0 → [], [d] ⧺ f (10p − w d, 10q − w d))`
$d = (10q)\ \mathtt{div}\ w$, $w = 2^{17}$
	the same in integer arithmetic only, as chapter 3 required of $\mathtt{intern}$: every interval reached is $(p/w,\ q/w)$